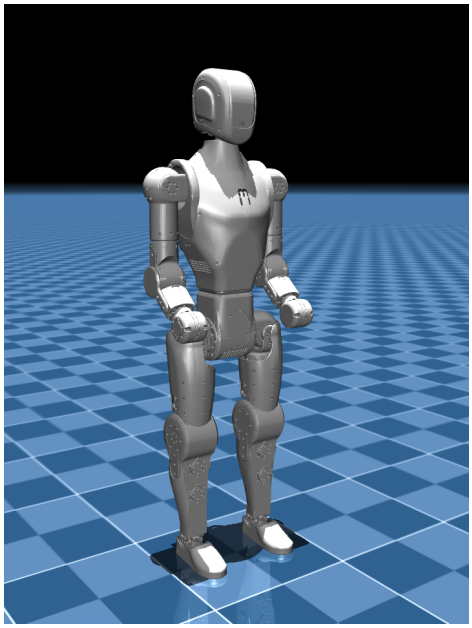
A HANDS-ON ROBOTICS LABORATORY

Engineering an AI Humanoid Robot

From first simulation to a safely commissioned physical research platform

Menlo Research Asimov 1

17 progressive lessons
Complete runnable teaching code
Numerical tests and measured results
ROS 2, kinematics, control and PPO
Perception and bounded skills
Staged EUR 6,000 hardware planning



Research edition / 25 October 2026

Prepared for an experienced software engineer beginning robotics. Platform facts are source-linked; exercise assumptions and physical deployment gates are explicit.

Contents

Part I - Engineering the project 4

1. Project vision and how to use this book . . . 4
2. Feasibility assessment and official-source audit . . . 5
3. Complete system architecture . . . 7
4. Dependency-aware roadmap . . . 9
5. Recommended stack and reproducibility policy . . . 10
6. Repository layout and ownership . . . 11

Part II - The simulation laboratory 13

Lesson 0 - Establish a reproducible environment . . . 13
Lesson 1 - My first observable humanoid simulation . . . 15
Lesson 2 - Joint control as a measurable feedback loop . . . 19
Lesson 3 - Sensors, telemetry and state estimation . . . 21
Lesson 4 - Frames, forward kinematics and inverse kinematics . . . 23

Part III - Middleware, balance, learning and perception 27

Lesson 5 - A modular ROS 2 humanoid workspace . . . 27
Lesson 6 - Static balance and the limits of a supported pose . . . 33
Lesson 7 - Reinforcement learning without hidden assumptions . . . 35
Lesson 8 - Standing, walking and turning as research milestones . . . 40
Lesson 9 - Perception and embodied behavior . . . 43

Part IV - From simulation evidence to physical hardware 46

Lesson 10 - Sim-to-real contracts, identification and rollback . . . 46
Lesson 11 - Build a defensible EUR 6,000 budget . . . 49

Lesson 12 - Mechanical design, fabrication and incremental assembly 53

Lesson 13 - Electrical and embedded engineering on a single joint 56

Lesson 14 - Assembly, calibration and supported commissioning 59

Part V - The end-to-end research demonstration **62**

Lesson 15 - Recognize an object and execute a validated skill 62

Lesson 16 - Research quality, maintenance and the next build 64

Appendices - Reproducibility and evidence **67**

Appendix A. Complete numerical and fault tests 67

Appendix B. Measured preparation results 68

Appendix C. Live telemetry and model-rendering utilities 71

Appendix D. Mermaid architecture sources 73

Appendix E. Units, equations and glossary 74

Appendix F. Source register and update policy 75

Part I - Engineering the project

1. Project vision and how to use this book

Build the robot in layers. First make a model observable. Then move a joint under feedback, recover known geometric relationships, measure behavior, and build software boundaries that reject bad commands. Only after those skills are repeatable should money and force enter the project. The physical outcome depends on verified parts, fabrication, and commissioning evidence.

The simulation target is the Menlo Research Asimov 1. The initial physical target is a guarded, supported module that can sense an object and execute a small, repeatable gesture. A complete walking humanoid is a later research objective with separate feasibility and safety gates. Success means you can explain a failure using measurements, reproduce a result from a pinned configuration, and decide when the evidence is insufficient to proceed.

This book is written for an experienced software engineer beginning robotics. You already understand interfaces and distributed systems. The new challenges are physical units, geometric frames, forces, contact, uncertainty, deadlines, and faults that can release stored energy. A software rollback cannot restore a stripped gearbox.

Each lesson follows A-O: purpose, concepts, physical relevance, architecture, prerequisites, implementation, complete code, walkthrough, expected result, validation, troubleshooting, exercise, hardware connection, completion checklist, and preview. Read one lesson, run its tests, and record the result before continuing. All original teaching code is printed in the book and also saved beside it under humanoid_lab/. External CAD and meshes remain in the upstream checkout because copying them into code listings would not make them reproducible or useful.

What has and has not been demonstrated

The included lab is an educational simulation stack. Headless physics, model contracts, small IK, synthetic perception, command-gate tests, and the supported demonstration were exercised during preparation. Exact results appear in the validation appendix. ROS 2 package files are provided but require a Jazzy installation for integration testing. A short PPO run validates the training plumbing; it does not demonstrate learned standing or walking. No physical actuator was connected, no harness was load tested, and no completed robot is claimed.

Verified means supported by a named primary source or directly inspected asset. Engineering assumption means a proposed design value that must be tested. Unverified means a claim not independently established here. Missing means unavailable in the inspected public release, not proof that the manufacturer does not possess it. Every acceptance threshold proposed by this book is an engineering exercise, not a manufacturer rating.

Your workstation and budget

You reported an Intel Core i7 and NVIDIA GeForce RTX, with an optional EUR 2,000-3,000 laptop purchase. The exact CPU generation, RAM, GPU, VRAM, operating system, and driver remain unknown. Start with CPU MuJoCo. Do not buy a laptop based on the RTX label alone: mobile GPUs differ in memory, power limits, cooling, and sustained throughput. Treat the possible laptop allowance separately from the EUR 6,000 robot budget until you decide otherwise.

Run the following inventory commands on Linux; they read information without changing drivers. On Windows, record the same facts from System Information and nvidia-smi. Save output in your experiment log, including OS version and free SSD space.

```
lscpu
free -h
cat /etc/os-release
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
df -h .
```

For the early single-model lessons, 16 GB RAM and a modern CPU are a practical planning target, not a measured minimum. Rendering needs a functioning graphics stack; physics can run headlessly. Measure simulated seconds per wall second before planning a large training job. A new laptop is unnecessary for learning coordinates, controllers, ROS messages, or small PPO tasks.

2. Feasibility assessment and official-source audit

The official README describes a roughly 1.2 m, 35 kg machine with 25 powered joints and advertises a USD 20,000 unassembled kit. That public asking price is not an EU landed quotation. It is enough to reject the claim that buying the published kit fits EUR 6,000. It does not determine the self-source cost. [S1]

The checked-out main revision is 96a6807c163e4e6cabdc124eefcc45b0ae2cb57a. The GitHub tree was inspected directly on 25 October 2026. Some search-indexed pages described an older 23-joint simulation and different paths. The actual pinned checkout contains 25-DOF and 27-DOF model variants. Use the 25-DOF version throughout this book. Its upstream simulation README names MuJoCo 3.4.0 as validated; the lab uses that version. [S2]

Subject	Evidence in this edition	Status and consequence
Mechanical source	mechanical/ASV1 contains 168 STEP files and 25 PDFs in the inspected tree	Verified file inventory; not assembly quantities
Electrical source	electrical/ contains board sources and wiring CSVs	Verified design assets; release reconciliation required
Simulation	sim-model/asimov_1_25dof/xml/asimov_1_25dof.xml and matching URDF	Verified files; loaded in the lab
27-DOF variant	Separate model with gripper structure	Verified variant; excluded from this curriculum
Actuation in 25-DOF MJCF	No actuator definitions in inspected XML	Adapter explicitly adds teaching torque motors
Firmware and SDK	Public manual describes operation while SDK is in development	Availability must be confirmed for purchased release
ROS 2 compatibility	No tested upstream ROS 2 package established in this checkout	Book supplies an educational bridge
Complete procurement BOM	Manual requests email for BOM; referenced fabrication manifest absent here	No complete priced BOM established

Subject	Evidence in this edition	Status and consequence
Structural hardware revision	Commit identifies assets, not an approved physical serial-number release	Match fabrication, electronics and firmware before ordering
Licenses	HARDWARE-LICENSE.txt and SOFTWARE-LICENSE.txt	Preserve notices and review applicable redistribution terms

The repository's hardware/software licenses are CERN-OHL-S-2.0 and GPL-2.0 as published. Original teaching code is separate from those assets; do not assume packaging it together changes upstream obligations. This book provides engineering guidance, not a legal interpretation of redistribution terms. [S1]

Joint topology and revision mismatches

The documented powered layout is two six-axis legs, two five-axis arms, waist yaw, and two neck axes. Shoulder pitch/roll/yaw, elbow, and wrist yaw make five per arm. Hip pitch/roll/yaw, knee, ankle pitch, and ankle roll make six per leg. The hardware ankle is a parallel linkage: two motor coordinates map to pitch and roll. Simulator joints are not automatically motor shaft coordinates. [S3]

A concrete mismatch matters more than the label 'digital twin': the inspected XML gives the left elbow an upper range near 2.09443 rad, whereas the mechanical-tour table gives a different bound. Tutorial code reads its bounds from its pinned XML. Physical bounds must come from the accepted hardware release and measured mechanism. Never copy either table blindly into the motor driver.

The mechanical-tour page itself flags some illustrations as outdated. Documentation also describes a 23-DOF locomotion configuration with locked neck axes. These statements can coexist with a 25-axis physical robot. Model size, controlled action count, and physical motor count are different quantities. Record all three in a deployment manifest. [S3, S4]

Development status and buildability

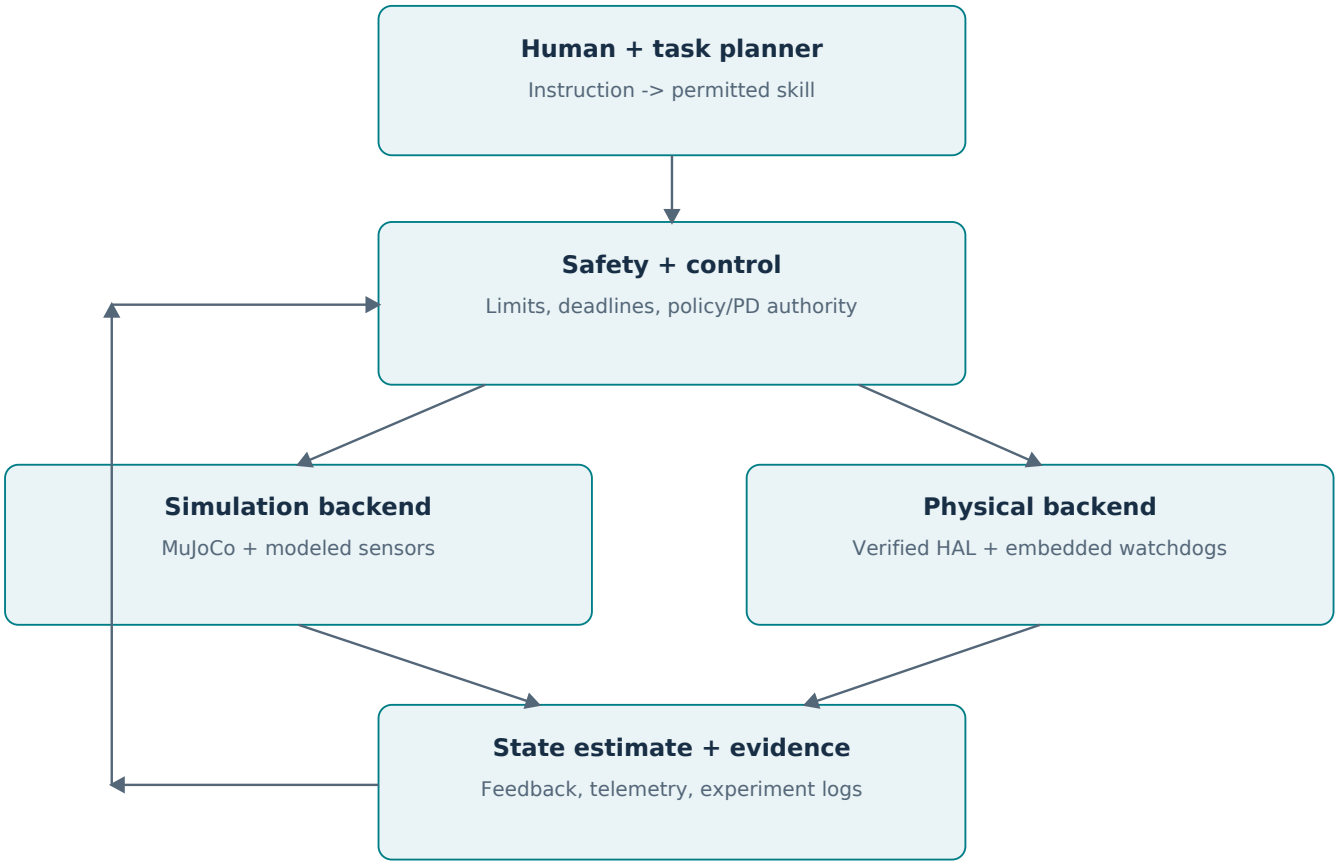
The public manual describes firmware modes and a supplied locomotion policy, while the repository roadmap lists software work still pending. Treat these as release-specific claims until the matching image, API schema, policy, checksums, calibration workflow, and support procedure are obtained. A manual's claim of zero-shot transfer is not evidence for your assembled unit. [S1, S4]

The platform is useful for simulation and inspecting real design tradeoffs. A full self-source build is not a beginner turnkey exercise: it combines machined parts, process-specific polymers, high-energy power, parallel ankle calibration, custom boards, and release-dependent software. Advance with one guarded module and engineering review of the support and power design.

The branch listing included main and several electrical, mechanical, gripper and simulation branches at inspection. Feature branches indicate work streams, not compatibility guarantees. Do not mix a board from one branch, an ankle from another, and firmware from a third without a documented integration decision.

3. Complete system architecture

Think of the system as a company with explicit authority. The planner proposes a job. A skills library translates it into a known procedure. A supervisor authorizes bounded execution. A controller makes small repeated corrections. Drivers deliver power and report measurements. No higher layer gets to bypass the lower layer's limits.



Subsystem	Analogy	Engineering function and interface
Structure and transmissions	Skeleton and tendons	Carry loads and define axis geometry, stiffness, backlash and travel
Actuators and drivers	Muscles and local reflexes	Produce limited torque; report encoder, current and thermal state
Encoders, IMU and camera	Senses	Measure different quantities in named frames with timestamps
Power and protection	Energy supply and circuit breakers	Distribute energy, limit faults and isolate hazardous motion
Embedded controller	Local reflex coordinator	Enforce deadlines, watchdogs and release-specific bus rules
Motion controller	Skilled operator	Track joint or whole-body targets within feasible constraints
State estimator	Shared situational picture	Fuse delayed, noisy observations with a model

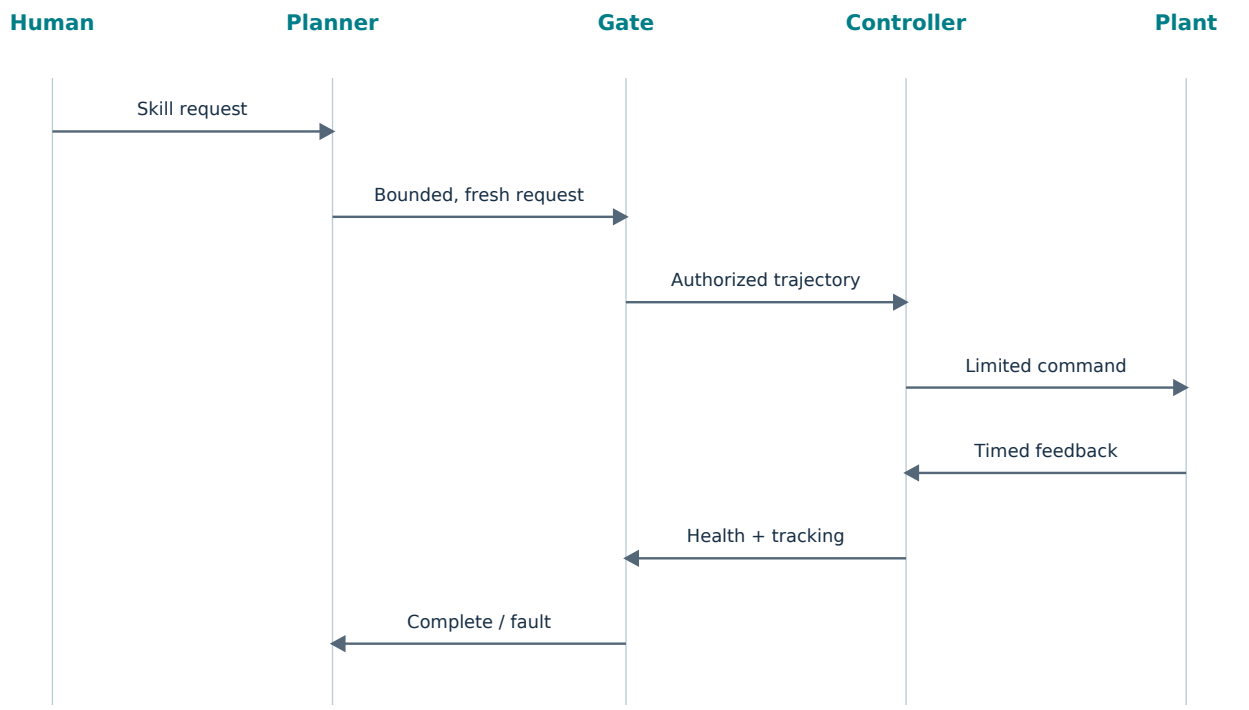
Subsystem	Analogy	Engineering function and interface
ROS 2	Typed internal messaging	Connect modular components; transport alone guarantees no control deadline
Simulator	Rehearsal space	Approximate dynamics, contacts, sensors and failure cases
Policy inference	Learned reflex	Map a defined observation to a bounded action at a defined rate
Perception and task planner	Eyes and foreman	Locate objects and select validated skills
Safety supervisor	Independent authority	Veto unsafe transitions, detect stale data and trigger a tested stop
Logging and deployment	Flight recorder and release management	Preserve evidence, hashes, configurations and rollback artifacts

The official electrical design divides media/networking and motion-control responsibilities across two compute stacks. Its documented CAN-interface specification and physical branch mapping are not fully reconciled publicly; the manual explicitly calls that out. Do not infer which baud rate belongs to a limb from the number of connectors. [S5]

Control, policy and high-level AI

Scripted control follows a prescribed time trajectory. Classical feedback uses error and a model, for example a proportional-derivative joint controller. A learned policy selects actions based on training experience. High-level AI selects a task or skill. Each can exist without the others. A wave following a sine function is not reinforcement learning, and a language interface does not make a motor controller intelligent.

Use simulation time for physical integration and monotonic wall time for watchdog deadlines. Keep these clocks explicit in messages. A playback run may execute ten simulated seconds in one wall second; a physical watchdog must still expire in actual elapsed time. A camera frame's exposure time is not its arrival time.



The physical power-isolation path must remain effective if ROS, the policy process, the planner, or the network hangs. Cutting motor power may cause a heavy robot to collapse; the support rig is part of the stop strategy. Software DAMP mode is not an emergency stop. The official software guide makes this distinction explicit. [S4]

4. Dependency-aware roadmap

Effort estimates below are planning ranges for an experienced software engineer, in focused hours. They include learning and debugging but exclude supplier lead times, fabrication queues, and recovery from hardware damage. Hardware costs remain unquoted unless specifically identified.

Phase	Deliverable and dependency	Effort	Acceptance gate
0	Reproducible environment; none	8-16 h	Import versions, save lock and hardware inventory
1	Asimov view and model inspection; 0	6-12 h	Correct joint count and finite state
2	Bounded joint trajectory; 1	12-24 h	Limits, error and saturation measured
3	Telemetry and sensor semantics; 2	12-24 h	Units, frames and time recorded
4	Forward/inverse kinematics; 2-3	20-40 h	Reachable target residual under 2 mm
5	ROS 2 workspace; 2-4	20-40 h	Two nodes communicate and stale command rejected
6	Balance experiments; 3-5	24-60 h	Repeatable unsupported evaluation and honest fall report
7	PPO task and export; 2-6	30-80 h	Held-out evaluation beats chosen baseline

Phase	Deliverable and dependency	Effort	Acceptance gate
8	Locomotion research; 6-7	80-200+ h	Tracking, falls, energy and slip evaluated
9	Perception and bounded skills; 4-5	20-50 h	Detect/no-detect cases and unknown-depth behavior pass
10	Sim-to-real contracts; 3-9	30-80 h	Observation/action/timing replay and rollback pass
11	Procurement and fabrication; 10 plus quotes	20-60 h	Complete release-matched landed BOM within budget
12	Single-actuator electrical bench; 11	30-80 h	Isolation, watchdog, feedback and limits measured
13	Incremental assembly; 12	40-120 h	One module at a time passes inspection
14	Supported commissioning; 13	40-100+ h	Signed records and repeatable stop tests
15	Physical skill deployment; 10 and 14	40-120+ h	Supported behavior repeatable without envelope violations
16	Integrated research demonstration; 9 and 15	20-60 h	Object-dependent skill, logs, faults and rollback demonstrated

Keep these dependencies visible. Procurement discovery can run while learning simulation, but purchasing high-value actuators before a compatible release is identified is premature. Locomotion is a research milestone, not a guaranteed outcome at a fixed training step count. Hardware phases may stop at a modular platform while simulation continues to full-body locomotion.

Use a milestone record with fields: gate, date, repository hash, environment lock hash, model hash, configuration, seed, test command, observed metric, threshold, outcome, reviewer, and artifact path. A failed result is useful evidence. Do not overwrite it with a later success using the same filename.

5. Recommended stack and reproducibility policy

The baseline is Ubuntu 24.04 LTS, Python 3.12, MuJoCo 3.4.0, and ROS 2 Jazzy. Jazzy remains supported through May 2029 according to the ROS release table. Lyrical is newer and listed through May 2031; it is not required for this curriculum. We choose Jazzy for its established Ubuntu 24.04 package set and Gazebo Harmonic pairing. Iron is already end-of-life. Recheck the release table when updating this book's environment. [S6, S7, S8]

Tool	Best role here	Hardware and integration tradeoff
MuJoCo 3.4.0	First simulation, dynamics, IK, CPU RL	Fast headless CPU physics; Python bridge needed for ROS; native MJCF
Gazebo Harmonic	Optional ROS-centric sensor and systems work	ROS Jazzy pairing documented; URDF/SDF conversion and plugin work; GPU rendering varies

Tool	Best role here	Hardware and integration tradeoff
Isaac Sim	Optional rich rendering and RTX sensor workflows	Significant RTX/VRAM and driver requirements; asset import must be revalidated
Isaac Lab	Optional batched training framework	A framework built around compatible simulation backends, not a drop-in MJCF viewer

No simulator is 'the most realistic' for every contact, actuator, camera, or deadline. Validate the phenomenon that matters: a soft toe, backlash, thermal derating, or latency may dominate your transfer error more than the solver brand. URDF captures a tree of links and joints; MJCF adds MuJoCo-specific world, contact, sensor and actuator settings. Conversion can lose semantics and requires regression tests.

Current Isaac Sim requirements list 32 GB RAM and 16 GB VRAM at the minimum tier, with an RTX 4080-class GPU and release-specific drivers. These are product requirements, not a promise that a similarly named laptop GPU performs like its desktop counterpart. Use the Compatibility Checker for the chosen release. Keep optional Isaac environments isolated from this book's CPU virtual environment; select an exact Isaac Lab/Sim/Python pair from their cross-product compatibility page after the hardware inventory is known. No unverified pair is installed by the tutorial. [S9]

Host installation gives direct device access and straightforward desktop rendering. A virtual environment isolates Python packages but shares the host driver and kernel. A container adds userspace reproducibility but still depends on host graphics, USB/CAN permissions and kernel timing. A virtual machine isolates the OS but may complicate graphics and device passthrough. Use native Ubuntu for the initial lab when available; a headless container is useful for CI after the baseline works.

The pinned Python packages are a tested teaching baseline rather than claims about the newest versions: MuJoCo 3.4.0, NumPy 2.2.6, Matplotlib 3.10.3, pytest 8.4.1, Gymnasium 1.2.1, Stable-Baselines3 2.7.0, OpenCV 4.12.0.88, and CPU PyTorch 2.8.0. A full resolved freeze is supplied beside the code. Save apt package versions separately because pip cannot record ROS Debian packages.

6. Repository layout and ownership

All tutorial commands assume the following location. If you use another location, update the shell root and HUMANOID_LAB_ROOT together. The actual book and code are saved at these paths.

```
/data/Robotics_Project/
  AI_Humanoid_Robotics_Engineering_Book.pdf
  book_source/                 editable teaching manuscript
  research/asimov-1/           pinned upstream CAD and models
  humanoid_lab/
    requirements.txt          direct teaching dependencies
    requirements-lock.txt     resolved environment snapshot
    model.py                  simulator adapter and bounded PD
    simulate.py               joint inspection and viewer
    telemetry.py              recorded plots and sensor inventory
    kinematics.py             planar FK and Asimov position IK
    balance.py                static geometry and passive fall baseline
    learning.py               reach/stand/walk Gymnasium tasks and PPO
    perception.py             image-space color detector
    safety.py                 offline command-gate exercise
    demo.py                   supported perception-to-gesture demo
    audit.py                  procurement and commissioning records
    config/                   BOM and commissioning records
```

```
tests/                    meaningful numerical and fault tests
ros2_ws/src/humanoid_bridge/  ROS node package and launch file
logs/                     experiment outputs; do not version large runs
```

The simulator adapter deliberately does not import a physical CAN library. That is an architectural boundary: the tests cannot accidentally send commands to a real actuator. Create a separate hardware transport only after the release-specific protocol is verified, with its own reviewed configuration and test fixtures.

Part II - The simulation laboratory

Lesson 0 - Establish a reproducible environment

A. Why this matters

You need to distinguish a model bug from a package mismatch. An environment you cannot recreate makes every later result weaker. Begin with a small, explicit dependency set and a commit-addressed robot model.

B. Concepts explained simply

A physics engine numerically advances a state. The state includes positions and velocities. A model contains geometry, mass, joints, and equations that govern those changes. Rendering only draws a state. Therefore a successful headless test proves something useful even when no window can open, but does not prove the graphics stack works.

C. Real-world relevance

A released robot controller needs traceable software, calibration and model versions. The same habit begins here with a Python environment and a git hash.

D. Architecture

Operating system -> Python virtual environment -> MuJoCo binding -> pinned robot model -> tests and logs. ROS is added later so the first failure has fewer possible causes.

E. Prerequisites

Ubuntu 24.04, internet access and adequate disk space. The commands below are for the reader's machine, not instructions to change an existing host blindly. Keep the supplied files at /data/Robotics_Project/humanoid_lab. If /data is not yours to write, choose a user-owned project directory and adapt paths consistently.

F. Step-by-step implementation

Install standard development and desktop OpenGL support, create a venv, then install CPU PyTorch before the other dependencies so pip does not select a large CUDA stack. sudo changes OS packages; understand and review those commands on your own machine.

```
sudo apt update
sudo apt install python3-venv python3-dev git build-essential libgl1 libglfw3
cd /data/Robotics_Project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cpu
python -m pip install -r humanoid_lab/requirements.txt
python -m pip check
python -m pip freeze > humanoid_lab/requirements-lock.txt
```

The supplied research checkout is already present. To recreate it on another machine where the directory does not exist:

```
cd /data/Robotics_Project
```

```
mkdir -p research
git clone https://github.com/menloresearch/asimov-1.git research/asimov-1
git -C research/asimov-1 checkout 96a6807c163e4e6cabdc124eefcc45b0ae2cb57a
git -C research/asimov-1 rev-parse HEAD
```

Set VS Code's interpreter to .venv/bin/python. Open the project root, run tests from a terminal first, and use a breakpoint before mj_step to inspect arrays. Do not combine a Conda interpreter, a ROS system interpreter and a pip venv accidentally.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/requirements.txt

/data/Robotics_Project/humanoid_lab/requirements.txt

```
mujoco==3.4.0
numpy==2.2.6
matplotlib==3.10.3
pytest==8.4.1
gymnasium==1.2.1
stable-baselines3==2.7.0
opencv-python-headless==4.12.0.88
```

H. Code walkthrough

Direct dependency pins make the intended API set explicit. The resolved lock records transitive versions as well. The upstream commit fixes asset paths and model semantics. A lock captured on x86 Linux is not necessarily portable to a different operating system or architecture.

I. Expected result

mujoco imports and prints 3.4.0. pip check reports no dependency conflicts. The checkout hash matches the printed revision. No robot hardware is involved.

J. Validation

```
python -c "import mujoco, numpy, torch; print(mujoco.__version__, numpy.__version__,
    torch.__version__)"
cd /data/Robotics_Project/humanoid_lab
python -m pytest -q
```

The test files are printed in the validation appendix. The prepared edition used Python 3.12.14; exact transitive versions are in requirements-lock.txt.

K. Troubleshooting

If wheels are unavailable, verify Python 3.12 and CPU architecture before compiling packages. If OpenGL fails, run headless physics first; a DISPLAY or driver error is distinct from an XML compilation failure. On macOS, passive-viewer scripts require mjpython; this book's ROS setup remains Ubuntu-specific. [S10]

L. Practical exercise

Create an environment manifest with OS, Python, model hash, package freeze and workstation inventory. Recreate a second venv from the freeze and compare imports.

M. Connection to the physical Asimov 1

The release-manifest habit transfers directly. These Python package pins do not select motor firmware or electrical revisions.

N. Completion checklist

You can explain the difference between a model, state and renderer; reproduce imports; identify your active Python executable; and locate the exact upstream model. Budget consumed by these software steps: EUR 0 excluding your existing computer and internet.

O. Next lesson preview

Load the humanoid, inspect its joint coordinates and observe a supported motion.

Lesson 1 - My first observable humanoid simulation

A. Why this matters

Seeing the robot is the first milestone, but observation must include names, coordinate addresses, limits and time. A picture alone cannot establish that commands reach the intended joint.

B. Concepts explained simply

A degree of freedom is an independent coordinate. A hinge joint rotates; a slide joint translates. A floating base adds three translations and a quaternion representation of orientation: its position array has seven entries while its velocity has six. Thus nq and nv can differ. Joint index, position-array address, velocity-array address and actuator index are different identifiers.

C. Real-world relevance

A wrong mapping can send a knee target to a shoulder. Map by verified names, then check dimensions and ordering before every deployment.

D. Architecture

Pinned MJCF -> teaching adapter -> MjModel -> MjData -> bounded controller -> mj_step -> viewer and CSV. The adapter fixes the pelvis, adds torque motors and retains the official mesh and inertial model.

E. Prerequisites

Lesson 0, the upstream meshes, model.py and simulate.py below. Your first run uses a fixed pelvis fixture. It is intentionally incapable of demonstrating free standing.

F. Step-by-step implementation

```
cd /data/Robotics_Project/humanoid_lab
python simulate.py --headless --seconds 10
python simulate.py --seconds 30
```

The first command tests physics without opening a window. The second opens a viewer with a useful camera distance. Rotate and zoom with the viewer controls and watch the left elbow. Head, torso and legs are visible even though this exercise moves one arm joint.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/model.py

/data/Robotics_Project/humanoid_lab/model.py

```
"""Educational Asimov model adapters; simulation only."""
from pathlib import Path
import xml.etree.ElementTree as ET
import mujoco as mj
import numpy as np

ROOT = Path(__file__).resolve().parent
ASSET = (ROOT.parent / 'research/asimov-1/sim-model/'
         'asimov_1_25dof/xml/asimov_1_25dof.xml')

def load(supported: bool = True):
    tree = ET.parse(ASSET)
    xml = tree.getroot()
    xml.find('compiler').set('meshdir', str(ASSET.parent.parent / 'meshes'))
    xml.find('option').set('timestep', '0.002')
    # Fixed pelvis in the teaching fixture. This is not balancing.
    if supported:
        base = xml.find('.//freejoint')
        xml.find('.//body[@name="pelvis_link"]').remove(base)
    for actuator in list(xml.findall('actuator')):
        xml.remove(actuator)
    actuators = ET.SubElement(xml, 'actuator')
    joints = xml.findall('.//worldbody//joint')
    for j in joints:
        # Remove passive springs so they do not hide controller behavior.
        for attr in ('stiffness', 'damping', 'springref'):
            j.set(attr, '0')
        limits = j.get('actuatorfrcrange', '-8 8')
        ET.SubElement(actuators, 'motor', name=j.get('name')+'_motor',
                      joint=j.get('name'), gear='1',
                      ctrllimited='true', ctrlrange=limits)
    model = mj.MjModel.from_xml_string(ET.tostring(xml, encoding='unicode'))
    model.opt.gravity[:] = [0, 0, -9.81]
    data = mj.MjData(model)
    mj.mj_forward(model, data)
    return model, data

def addresses(model):
    ids = model.actuator_trnid[:, 0].astype(int)
    return ids, model.jnt_qposadr[ids], model.jnt_dofadr[ids]

def names(model):
    ids, _, _ = addresses(model)
    return [mj.mj_id2name(model, mj.mjtObj.mjOBJ_JOINT, int(j)) for j in ids]

def command(model, data, target, kp=35., kd=4., fraction=0.25):
    ids, qi, vi = addresses(model)
    target = np.asarray(target, dtype=float)
    if target.shape != (model.nu,) or not np.isfinite(target).all():
        raise ValueError('Expected one finite target per actuator')
    lo, hi = model.jnt_range[ids].T
    target = np.clip(target, lo, hi)
    # qfrc_bias is ideal model feedforward, not measured motor torque.
    torque = kp*(target-data.qpos[qi])-kd*data.qvel[vi]+data.qfrc_bias[vi]
    cap = np.max(np.abs(model.actuator_ctrlrange), axis=1)*fraction
    data.ctrl[:] = np.clip(torque, -cap, cap)
    return target.copy()
```

File: /data/Robotics_Project/humanoid_lab/simulate.py

/data/Robotics_Project/humanoid_lab/simulate.py

```
"""Inspect, view and log a supported Asimov joint trajectory."""
import argparse
import csv
import time
import numpy as np
import mujoco as mj
from model import ROOT, load, addresses, names, command

def main():
    p = argparse.ArgumentParser()
    p.add_argument('--headless', action='store_true')
    p.add_argument('--seconds', type=float, default=10.)
    args = p.parse_args()
    if not 0 < args.seconds <= 600:
        p.error('seconds must be in (0, 600]')
    m, d = load()
    ids, qi, vi = addresses(m)
    joint_names = names(m)
    print('nq, nv, nu:', m.nq, m.nv, m.nu)
    for name, j in zip(joint_names, ids):
        print(name, 'range=', m.jnt_range[j])
    j = joint_names.index('left_elbow_joint')
    home = d.qpos[qi].copy()
    path = ROOT/'logs/joints.csv'
    path.parent.mkdir(exist_ok=True)
    with path.open('w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['t', 'target', 'q', 'dq', 'command_Nm'])
        def tick():
            target = home.copy()
            target[j] += 0.1*np.sin(2*np.pi*0.25*d.time)
            actual = command(m, d, target)
            mj.mj_step(m, d)
            if not np.isfinite(d.qpos).all():
                raise RuntimeError('Nonfinite simulation state')
            writer.writerow([d.time, actual[j], d.qpos[qi[j]],
                             d.qvel[vi[j]], d.ctrl[j]])
            if round(d.time/m.opt.timestep) % 100 == 0:
                f.flush()
        if args.headless:
            for _ in range(round(args.seconds/m.opt.timestep)):
                tick()
        else:
            import mujoco.viewer
            with mujoco.viewer.launch_passive(m, d) as v:
                v.cam.distance = 2.2
                v.cam.lookat[:] = [0, 0, .65]
                v.cam.elevation = -15
                while v.is_running() and d.time < args.seconds:
                    start = time.monotonic()
                    tick()
                    v.sync()
                    time.sleep(max(0., m.opt.timestep-(time.monotonic()-start)))
    print('Saved', path)

if __name__ == '__main__':
    main()
```

H. Code walkthrough

load reads the verified XML rather than reconstructing a humanoid from guessed geometry. An absolute mesh directory makes the in-memory XML resolve assets correctly. Removing the floating joint attaches the pelvis to the world for this teaching fixture. The adapter replaces actuator definitions with gear-one torque motors, so ctrl is a simulated torque command in Nm. This is an educational actuator model, not an identified physical drive.

addresses obtains each actuator's joint and its qpos/qvel addresses. command checks the entire target vector, clips positions to model bounds, computes feedback plus ideal bias compensation, and clips torque to 25% of the XML actuator control envelope. The 25% value is a simulation teaching choice. mj_step integrates a 2 ms timestep; the CSV stores each step's feedback. The viewer's sync updates graphics; the sleep aims at readable wall-clock speed but provides no hard deadline.

I. Expected result

The supported model has nq=25, nv=25 and nu=25. The terminal lists joint ranges, a small elbow motion appears in the viewer, and logs/joints.csv contains about 5,000 data rows for ten simulated seconds.

J. Validation

Run test_asimov_contract and test_bounded_feedback_and_rejection from the appendix. Check finite state, model gravity, time increment and the printed actuator count. Manually verify that the left elbow moves while the fixed pelvis stays fixed. Share the test output before proceeding to Lesson 2 if following this as a coached course.

K. Troubleshooting

A missing mesh often means the checkout is incomplete or paths belong to an older release. An absent actuator means the original model was opened without the adapter. A falling robot usually means you loaded the free-base model directly; that is not this fixture. If the window does not open, preserve the successful headless result and diagnose graphics separately.

L. Practical exercise

Print joint type, qpos address and qvel address in a four-column inventory. Compare the supported and unsupported model dimensions without applying a motor command.

M. Connection to the physical Asimov 1

Names and geometric inspection are reusable. Fixing the pelvis, ideal bias compensation, low-delay observations and gear-one torque motors are simulator conveniences that must be replaced or calibrated.

N. Completion checklist

You can launch both modes, identify all 25 actuators, explain nq versus nv, find the CSV, and state why this demonstration proves no balance capability. No hardware expenditure is required.

O. Next lesson preview

Measure tracking and introduce feedback gains, saturation and safe trajectory shapes.

Lesson 2 - Joint control as a measurable feedback loop

A. Why this matters

An angle command is a request. Physical motion depends on torque, inertia, gravity, friction, limits and time. Tracking error tells you whether the request is achieved.

B. Concepts explained simply

Position q is an angle in radians, velocity dq is its rate in rad/s, and torque is a turning effort in Nm. One radian is about 57.3 degrees. A revolute joint changes orientation; a prismatic joint changes distance. The same controller structure can apply, but its units change.

Our controller uses tau = Kp*(q_target-q) - Kd*dq + bias(q,dq). Kp has units Nm/rad and Kd has units Nm*s/rad. With error 0.1 rad, Kp=35 and velocity 0.2 rad/s, Kd=4, the feedback portion is 3.5-0.8=2.7 Nm. Gravity and other bias terms are additional. A torque cap may reduce the result.

PID adds Ki times the accumulated error. Integral action can remove steady offsets but can wind up when the motor saturates. Begin with PD and an identified bias before adding an integral. If using integral control, freeze or back-calculate the integral during saturation and reset it on state transitions.

C. Real-world relevance

Actual current, temperature, voltage and tracking-error measurements constrain usable gains. A high simulated gain does not authorize the same gain on an assembled limb.

D. Architecture

Reference trajectory -> position/velocity limits -> PD and bias -> torque saturation -> actuator and joint -> encoder feedback. Log both requested and applied values; hiding clipping prevents diagnosis.

E. Prerequisites

Lesson 1 and basic differentiation. Keep the fixed-pelvis fixture. The code for this lesson is the complete command function and simulator from Lesson 1; no unprinted source file is required.

F. Step-by-step implementation

Run the ten-second trajectory, then render its telemetry. Change only one simulation parameter at a time. Try Kp=20 with Kd=4, then Kp=35 with Kd=6. Preserve separate CSVs before rerunning.

```
python simulate.py --headless --seconds 10
cp logs/joints.csv logs/joints_baseline.csv
python telemetry.py
```

For both arms, create a second index by joint name and use a mirrored bounded elbow excursion. For a knee, read its asymmetric sign and limits first. Test a 0.03 rad knee excursion in the supported fixture, never by assuming left and right signs match. A minimum-jerk transition uses s(t)=10*r^3-15*r^4+6*r^5, r clipped to [0,1], with q(t)=q_start+s*(q_end-q_start).

G. Complete code

The executable files are model.py and simulate.py printed in Lesson 1, plus telemetry.py in Lesson 3. The additional numerical analysis is this complete terminal program:

```
python - <<'PY'
import numpy as np
x = np.genfromtxt('logs/joints.csv', delimiter=',', names=True)
e = x['target']-x['q']
print('RMSE rad:', np.sqrt(np.mean(e*e)))
print('Peak speed rad/s:', np.max(np.abs(x['dq'])))
print('Peak command Nm:', np.max(np.abs(x['command_Nm'])))
assert np.isfinite(e).all()
PY
```

H. Code walkthrough

The sine target is 0.1 rad at 0.25 Hz around the XML reference pose, with model-limit clipping. The callback executes before integration so torque is computed from the current state. qfrc_bias is ideal model information; physical software would need a trusted dynamics model or another strategy. Position and torque clipping protect different quantities. Neither ensures collision avoidance.

I. Expected result

The elbow follows a smooth curve with nonzero error. Torque may plateau where the simulation cap is active. The supported pelvis prevents whole-body falling but other joints can still oscillate or contact.

J. Validation

Require finite state and applied commands within their configured caps. Proposed exercise target: elbow RMSE below 0.1 rad for the baseline run; record the measured value rather than assuming success. Inject NaN and wrong-length targets and verify rejection. Compare a smooth trajectory against a step command in simulation and inspect peak speed.

K. Troubleshooting

Oscillation suggests excessive gain, inadequate damping or an integration issue. Persistent error with a flat-topped torque trace suggests saturation or load mismatch. A joint moving opposite to expectation suggests a sign/frame problem, not a need to negate every command globally.

L. Practical exercise

Implement minimum-jerk motion as a new simulation-only trajectory, compute peak speed, and reject duration choices that exceed a chosen speed envelope.

M. Connection to the physical Asimov 1

The feedback structure and logging transfer. Gains, friction, current-to-torque conversion, backlash, position zeros, thermal limits and ankle mapping require measured data. Simulator limits are not physical protection settings.

N. Completion checklist

You can distinguish angle, speed and torque; reproduce tracking metrics; explain saturation and integral windup; and reject invalid commands. Continue only after a repeatable joint experiment.

O. Next lesson preview

Read sensors with explicit units, frames and timestamps, and turn logs into a useful diagnostic view.

Lesson 3 - Sensors, telemetry and state estimation

A. Why this matters

A controller needs a trustworthy estimate of what the robot is doing now. Sensor messages are partial, noisy, delayed observations, not the entire physical state.

B. Concepts explained simply

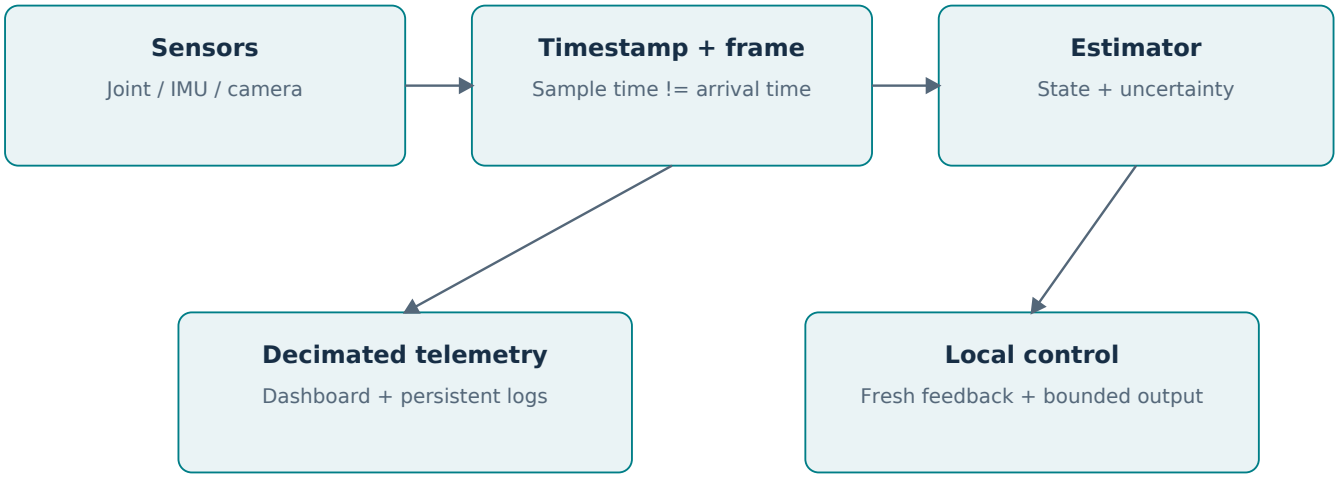
An encoder measures a shaft or output angle. A gyroscope measures angular velocity. An accelerometer measures specific force rather than simply the second derivative of world position; its stationary reading includes the support reaction against gravity. A camera measures light projected into pixels. Foot contact is an interaction between surfaces, not a boolean always available from the real hardware.

Latency is age, jitter is variation in age, and sample frequency is how often a measurement is produced. A 10 Hz telemetry feed cannot be silently substituted for a 500 Hz feedback loop. Store exposure/sample time, receive time, sequence, frame and units. Your IoT experience helps with delivery and observability, while actuator deadlines require a different timing contract.

C. Real-world relevance

The physical robot's encoder offsets and IMU orientation affect every estimate. Gravity projected into the wrong frame can look like a sustained acceleration. Motor current is not a calibrated contact-force sensor.

D. Architecture



E. Prerequisites

A successful joint log from Lesson 2, NumPy and Matplotlib. No physical IMU or camera is needed for this lesson.

F. Step-by-step implementation

```
python telemetry.py
```

Open logs/telemetry.png. The terminal prints sensor names and dimensions from the actual model, then its subtree center of mass in world coordinates. MuJoCo gyro and accelerometer channels are modeled sensors; frame quaternion and linear velocity channels are privileged simulation signals unless corresponding estimators exist on hardware.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/telemetry.py

/data/Robotics_Project/humanoid_lab/telemetry.py

```
"""Plot recorded feedback; no actuator network connection."""
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from model import ROOT, load

if __name__ == '__main__':
    data = np.genfromtxt(ROOT/'logs/joints.csv', delimiter=',', names=True)
    fig, axes = plt.subplots(3, 1, figsize=(8, 7), sharex=True)
    axes[0].plot(data['t'], data['target'], label='target')
    axes[0].plot(data['t'], data['q'], label='encoder truth')
    axes[0].set_ylabel('Angle [rad]'); axes[0].legend()
    axes[1].plot(data['t'], data['dq']); axes[1].set_ylabel('Speed [rad/s]')
    axes[2].plot(data['t'], data['command_Nm']); axes[2].set_ylabel('Command [Nm]')
    axes[2].set_xlabel('Simulation time [s]')
    fig.tight_layout(); fig.savefig(ROOT/'logs/telemetry.png', dpi=160)
    m, d = load()
    for i in range(m.nsensor):
        a, n = m.sensor_adr[i], m.sensor_dim[i]
        print(m.sensor(i).name, d.sensordata[a:a+n].copy())
    print('COM world [m]:', d.subtree_com[m.body('pelvis_link').id])
```

H. Code walkthrough

genfromtxt reads named CSV columns. Three aligned plots share time so you can relate error, velocity and saturation. Sensor addresses and dimensions select exact slices; copying samples avoids retaining mutable simulator views. subtree_com is mass-weighted model truth and is used later to check an estimator, not as a sensor you can plug into a real robot.

For a basic attitude estimator, integrate gyro orientation over small dt, then correct roll/pitch slowly toward an accelerometer-derived gravity direction only when linear acceleration is plausibly low. A complementary filter combines fast gyro changes with slow gravity reference. Accelerometers alone cannot determine yaw, and a six-axis IMU without external references drifts in yaw. A full estimator also uses kinematic contact constraints and rejects slipping-foot assumptions.

I. Expected result

Readable angle, speed and torque-command plots. The initial supported fixture's sensor values can differ from expectations until you identify local axes and specific-force conventions. Investigate a discrepancy rather than relabeling the plotted units.

J. Validation

Check CSV time is monotonic with approximately 0.002 s increments. Verify the encoder channel's unit by changing a joint by 0.1 rad. For an added noise model, use a fixed random seed and show both ground truth and noisy measurement. Verify quaternion ordering before comparing to ROS: MuJoCo uses w,x,y,z while ROS messages use x,y,z,w.

K. Troubleshooting

A constant copied history often comes from storing array views. A rotated gravity vector may indicate world versus sensor-frame confusion. A slow plot may indicate every physics step is being rendered. Decimate visualization while preserving controller sampling.

L. Practical exercise

Add synthetic gyro bias and Gaussian encoder noise to copied data only. Estimate a zero-rate gyro bias from a stationary segment, subtract it, and compare drift over a held-out segment.

M. Connection to the physical Asimov 1

Telemetry schemas and diagnostic plots are reusable. Sensor timing, calibration, transport and noise must come from real recordings. Do not assume foot force sensing is present merely because the simulator reports contacts.

N. Completion checklist

Every stream has known units and frame; every time variable has a clock; plotted values are copies; and ground truth is distinguished from measurements and estimates.

O. Next lesson preview

Use geometry to predict and command limb endpoints.

Lesson 4 - Frames, forward kinematics and inverse kinematics

A. Why this matters

A hand target is usually expressed in space, while motors accept joint coordinates. Kinematics translates between those descriptions before dynamics determines the forces required.

B. Concepts explained simply

A coordinate frame is an origin and three axes. A rotation changes the basis used to express a vector; a translation changes its origin. A 90-degree rotation about z maps (1,0,0) to (0,1,0). With a translation (1,2,0), that same point becomes (1,3,0). The order matters.

A homogeneous transform T has a 3x3 rotation R and translation p in its upper rows, and [0,0,0,1] below. It maps a homogeneous point using x_A=T_AB*x_B. Chaining transforms means multiplication in frame order. Inverting a rigid transform uses R transpose and -R transpose times p.

Euler angles are three ordered rotations and have singular configurations. A unit quaternion represents rotation with four numbers and a normalization constraint; q and -q represent the same orientation. Use matrices or quaternions for composition and reserve Euler angles for labeled displays.

For a two-link planar arm, x=l1*cos(q1)+l2*cos(q1+q2), y=l1*sin(q1)+l2*sin(q1+q2). With l1=.3 m, l2=.2 m and zero angles, the endpoint is (.5,0) m. At q1=pi/2 and q2=0 it is (0,.5) m. This is forward kinematics.

The Jacobian J is the local sensitivity of endpoint position to joint changes. For example, at zero angles the second row is [.5,.2], so a small elbow motion of .01 rad changes y by about .002 m. Fully stretched arms lose some directions of instantaneous motion: this is a singularity.

C. Real-world relevance

A camera-to-base transform and encoder zero error can dominate reaching accuracy. Kinematics must use the same joint conventions as the assembled mechanism.

D. Architecture

Target in world frame -> FK and Jacobian -> damped least-squares update -> joint constraints -> recompute FK -> residual. This lesson solves endpoint position only; it does not plan collision-free dynamic motion.

E. Prerequisites

Lessons 1-3, NumPy matrix multiplication and the fixed-pelvis fixture. No new kinematics library is needed for the first experiment. Pinocchio becomes useful for larger model-based dynamics and optimization workflows once your conventions are tested.

F. Step-by-step implementation

```
python kinematics.py
```

The target is a small displacement from the model's current left wrist-link origin. That choice makes reachability plausible and avoids inventing a hand geometry. To visualize, call solve before launching a MuJoCo passive viewer and inspect the resulting pose without dynamics. Kinematic pose inspection must not be mistaken for a physically executed trajectory.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/kinematics.py

/data/Robotics_Project/humanoid_lab/kinematics.py

```
"""Position-only damped least-squares IK in a fixed pelvis fixture."""
import numpy as np
import mujoco as mj
from model import load, addresses, names

def planar(q, lengths=(.3, .2)):
    a, b = q; l1, l2 = lengths
    return np.array([l1*np.cos(a)+l2*np.cos(a+b),
                     l1*np.sin(a)+l2*np.sin(a+b)])

def solve(m, d, target, iterations=150, tolerance=.002):
    joint_names = names(m)
    arm = [i for i, name in enumerate(joint_names) if name.startswith('left_')
           and any(x in name for x in ('shoulder', 'elbow', 'wrist'))]
    ids, qi, vi = addresses(m)
    bid = m.body('left_wrist_yaw_link').id
    jp, jr = np.zeros((3, m.nv)), np.zeros((3, m.nv))
    for _ in range(iterations):
        mj.mj_forward(m, d)
        error = target-d.xpos[bid]
        if np.linalg.norm(error) < tolerance:
            return True, float(np.linalg.norm(error))
        mj.mj_jacBody(m, d, jp, jr, bid)
        j = jp[:, vi[arm]]
        dq = j.T @ np.linalg.solve(j@j.T+.01**2*np.eye(3), error)
        d.qpos[qi[arm]] += np.clip(dq, -.04, .04)
        d.qpos[qi[arm]] = np.clip(d.qpos[qi[arm]],
            m.jnt_range[ids[arm], 0], m.jnt_range[ids[arm], 1])
    mj.mj_forward(m, d)
    return False, float(np.linalg.norm(target-d.xpos[bid]))

if __name__ == '__main__':
    print('Two-link at zero:', planar([0., 0.]))
    m, d = load()
    target = d.xpos[m.body('left_wrist_yaw_link').id].copy()
    target += [.02, 0., .01]
    ok, error = solve(m, d, target)
    print('IK converged:', ok, 'residual [m]:', error)
    if not ok:
        raise SystemExit('Target not reached; retain last validated pose')
```

The following complete numerical example verifies the frame convention:

```
python - <<'PY'
import numpy as np
R = np.array([[0., -1., 0.], [1., 0., 0.], [0., 0., 1.]])
p = np.array([1., 2., 0.])
point = R @ np.array([1., 0., 0.])+p
print(point)
assert np.allclose(point, [1., 3., 0.])
assert np.allclose(R.T @ R, np.eye(3))
PY
```

H. Code walkthrough

mj_forward updates poses for the current coordinates. mj_jacBody calculates the wrist body-origin Jacobian. The solver selects only left-arm columns, so leg and base coordinates cannot satisfy the hand request accidentally. It uses dq=J.T*(J*J.T+lambda^2*I)^-1*error with lambda=.01, caps each iteration at .04 rad, then clips joint coordinates. Damping reduces ill-conditioning but does not make unreachable targets reachable.

The result reports a boolean and residual, so a failed solve is not passed downstream as a completed task. This implementation mutates the candidate data even on failure; keep a separate validated state and copy it back on a physical planning failure. Add orientation residuals, collision checks and a trajectory stage before any dynamic reaching deployment.

I. Expected result

The planar example prints [.5,0]. The Asimov candidate reaches the nearby position within a 2 mm numerical tolerance on the fixed base. Numerical tolerance is not a claim of physical accuracy.

J. Validation

Use test_planar_geometry_and_ik. Add a finite-difference Jacobian check at a nonsingular pose: perturb one angle by epsilon, recompute the wrist position, divide its displacement by epsilon, and compare to the corresponding Jacobian column. For a target several meters away, require failure reporting rather than apparent success.

K. Troubleshooting

Wrong Jacobian columns create unexpected body motion. A stubborn residual near a limit may be reachability, not numerical failure. A body-origin target differs from a hand-tool tip; add a named site and include its offset before calling that target a grasp point.

L. Practical exercise

Implement a transform inversion test and a small Cartesian path of five reachable waypoints, each solved from the previous successful pose. Stop on the first failure and preserve its residual.

M. Connection to the physical Asimov 1

FK and frame tests are reusable after calibration. Wrist tool offsets, base attitude, link dimensions and the parallel ankle mapping require physical identification.

N. Completion checklist

You can transform a point, explain rotation order and quaternion ordering, validate a Jacobian column, distinguish reachability from convergence, and reject failed IK.

O. Next lesson preview

Place simulation and commands behind ROS 2 interfaces without changing the numerical controller.

Part III - Middleware, balance, learning and perception

Lesson 5 - A modular ROS 2 humanoid workspace

A. Why this matters

A single script is useful for proving a numerical idea. Multiple reusable robot capabilities need discoverable messages, explicit lifecycles and diagnostics. Keep the controller independent from the messaging layer so communication changes do not rewrite the physics.

B. Concepts explained simply

A node is a process participant. A topic carries a stream; a service is a request and response; an action is a long-running goal with feedback and cancellation. Parameters configure nodes. Launch files start a repeatable set of processes. QoS expresses delivery and buffering choices; reliable delivery can still deliver stale data, while best-effort delivery can lose a fresh packet.

TF2 maintains timestamped frame relationships. URDF describes the link tree consumed by robot_state_publisher and RViz2. JointState supplies scalar joint feedback but does not define frame geometry. ros2_control standardizes controller and hardware interfaces; it is not automatically integrated merely because a topic is named joint_states. Lifecycle nodes add explicit configuration and activation transitions, useful when enabling hardware only after prerequisites pass.

C. Real-world relevance

Separate simulation bridge, state estimation, kinematics, perception, behavior, safety and diagnostics nodes. Physical drivers should export verified state/command interfaces and preserve watchdog authority locally. The sample below provides a small two-node simulation system rather than claiming an entire hardware-control framework.

D. Architecture

wave node -> /joint_targets -> simulation bridge -> bounded PD and physics -> /joint_states -> wave initialization and telemetry. A future estimator consumes IMU/joint/contact streams, a planner produces validated paths, and a safety node gates activation. No planner bypasses the local limits.

E. Prerequisites

Ubuntu 24.04 and ROS 2 Jazzy. The package uses Python 3.12 to match Jazzy's system bindings. A separate --system-site-packages venv allows apt-installed rclpy to be imported. Do not activate an unrelated Conda environment.

F. Step-by-step implementation

The complete installer below follows the official apt-source bootstrap in [S7] and refuses other OS versions. It downloads the current official Noble repository-configuration package; this is a changing bootstrap dependency, so preserve its version and checksum with your environment record. Review the script before running it on your machine. It was not executed during book preparation.

/data/Robotics_Project/humanoid_lab/setup_ros.sh

```
#!/usr/bin/env bash
set -euo pipefail
. /etc/os-release
if [ "${ID:-}" != "ubuntu" ] || [ "${VERSION_ID:-}" != "24.04" ]; then
  echo 'This installer is for Ubuntu 24.04 only.' >&2
  exit 1
fi
sudo apt update
sudo apt install -y software-properties-common curl python3-venv
sudo add-apt-repository -y universe
python3 - <<'PY'
import json
import urllib.request
api = 'https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest'
release = json.load(urllib.request.urlopen(api))
assets = [x for x in release['assets'] if x['name'].startswith('ros2-apt-source_')
          and x['name'].endswith('.noble_all.deb')]
if len(assets) != 1:
    raise SystemExit('No unique Noble ROS 2 apt source asset; inspect official releases')
url = assets[0]['browser_download_url']
if not url.startswith('https://github.com/ros-infrastructure/ros-apt-source/releases/'):
    raise SystemExit('Unexpected download origin')
print('Official asset:', url)
urllib.request.urlretrieve(url, '/tmp/robotics-ros2-apt-source.deb')
PY
sudo dpkg -i /tmp/robotics-ros2-apt-source.deb
sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools python3-colcon-common-extensions
echo 'Next: source /opt/ros/jazzy/setup.bash'

bash /data/Robotics_Project/humanoid_lab/setup_ros.sh
```

After installation, prepare the package environment:

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools python3-colcon-common-extensions
source /opt/ros/jazzy/setup.bash
/usr/bin/python3 -m venv --system-site-packages /data/Robotics_Project/.venv-ros
source /data/Robotics_Project/.venv-ros/bin/activate
python -m pip install mujoco==3.4.0 numpy==2.2.6
export HUMANOID_LAB_ROOT=/data/Robotics_Project/humanoid_lab
cd /data/Robotics_Project/humanoid_lab/ros2_ws
colcon build --symlink-install --packages-select humanoid_bridge
source install/setup.bash
ros2 launch humanoid_bridge lab.launch.py
```

In a second terminal, source ROS and the workspace, then run ros2 topic echo /joint_states and ros2 topic hz /joint_states. Stop the wave node and observe the simulation fallback. The launch defaults to a GUI; for a headless integration check run the bridge executable directly without enabling its viewer parameter.

G. Complete code

All files below are in /data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/. Create resource/humanoid_bridge and humanoid_bridge/__init__.py as empty files. Names here are tutorial package names, not purported upstream Asimov packages.

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>humanoid_bridge</name><version>0.1.0</version>
  <description>Supported Asimov teaching bridge; simulation only.</description>
  <maintainer email="student@example.org">Robotics Lab</maintainer>
  <license>MIT</license>
  <buildtool_depend>ament_python</buildtool_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>sensor_msgs</exec_depend>
  <exec_depend>launch</exec_depend>
  <exec_depend>launch_ros</exec_depend>
  <export><build_type>ament_python</build_type></export>
</package>
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/setup.py

```
from setuptools import setup
setup(name='humanoid_bridge', version='0.1.0',
      packages=['humanoid_bridge'],
      data_files=[('share/ament_index/resource_index/packages', ['resource/humanoid_bridge']),
                  ('share/humanoid_bridge', ['package.xml']),
                  ('share/humanoid_bridge/launch', ['launch/lab.launch.py'])],
      install_requires=['setuptools'], zip_safe=True,
      maintainer='Robotics Lab', maintainer_email='student@example.org',
      description='Simulation teaching bridge', license='MIT',
      entry_points={'console_scripts': [
          'bridge = humanoid_bridge.bridge:main',
          'wave = humanoid_bridge.wave:main']})
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/setup.cfg

```
[develop]
script_dir=$base/lib/humanoid_bridge
[install]
install_scripts=$base/lib/humanoid_bridge
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/humanoid_bridge/bridge.py

```
import os
import sys
import time
sys.path.insert(0, os.environ['HUMANOID_LAB_ROOT'])
import mujoco as mj
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
from model import load, addresses, names, command

class Bridge(Node):
    def __init__(self):
        super().__init__('asimov_simulation')
        self.m, self.d = load()
        _, self.qi, self.vi = addresses(self.m)
        self.joints = names(self.m)
        self.home = self.d.qpos[self.qi].copy()
        self.target = self.home.copy()
        self.last = -np.inf
        self.pub = self.create_publisher(JointState, '/joint_states', 10)
        self.create_subscription(JointState, '/joint_targets', self.receive, 10)
        self.viewer = None
        if self.declare_parameter('viewer', False).value:
            import mujoco.viewer
            self.viewer = mujoco.viewer.launch_passive(self.m, self.d)
            self.viewer.cam.distance = 2.2
            self.viewer.cam.lookat[:] = [0, 0, .65]
        self.create_timer(.01, self.tick)

    def receive(self, msg):
        stamp = msg.header.stamp.sec+msg.header.stamp.nanosec*1e-9
        age = self.get_clock().now().nanoseconds*1e-9-stamp
        if msg.name != self.joints or len(msg.position) != self.m.nu:
            self.get_logger().warning('Rejected names or shape'); return
        if not 0 <= age <= .1 or not np.isfinite(msg.position).all():
            self.get_logger().warning('Rejected stale/nonfinite command'); return
        self.target = np.array(msg.position)
        self.last = time.monotonic()

    def tick(self):
        # Simulation fallback only. A physical machine needs a rig-specific stop.
        target = self.target if time.monotonic()-self.last < .1 else self.home
        for _ in range(5):
            command(self.m, self.d, target); mj.mj_step(self.m, self.d)
        msg = JointState(); msg.header.stamp = self.get_clock().now().to_msg()
        msg.name = self.joints
        msg.position = self.d.qpos[self.qi].tolist()
        msg.velocity = self.d.qvel[self.vi].tolist()
        msg.effort = self.d.actuator_force.tolist()
        self.pub.publish(msg)
        if self.viewer:
            self.viewer.sync()


def main():
    rclpy.init(); node = Bridge()
    try:
        rclpy.spin(node)
    finally:
        if node.viewer: node.viewer.close()
        node.destroy_node(); rclpy.shutdown()
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/humanoid_bridge/wave.py

```
import math
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class Wave(Node):
    def __init__(self):
        super().__init__('supported_elbow_wave')
        self.pub = self.create_publisher(JointState, '/joint_targets', 1)
        self.names = None; self.home = None; self.start = None
        self.create_subscription(JointState, '/joint_states', self.receive, 10)
        self.create_timer(.02, self.tick)

    def receive(self, msg):
        if self.home is None:
            self.names = list(msg.name); self.home = list(msg.position)
            self.start = self.get_clock().now().nanoseconds*1e-9

    def tick(self):
        if self.home is None: return
        now = self.get_clock().now()
        t = now.nanoseconds*1e-9-self.start
        msg = JointState(); msg.header.stamp = now.to_msg()
        msg.name = self.names; msg.position = self.home.copy()
        j = self.names.index('left_elbow_joint')
        msg.position[j] += .05*(1-math.cos(2*math.pi*.25*t))
        self.pub.publish(msg)

def main():
    rclpy.init(); node = Wave()
    try: rclpy.spin(node)
    finally:
        node.destroy_node(); rclpy.shutdown()
```

/data/Robotics_Project/humanoid_lab/ros2_ws/src/humanoid_bridge/launch/lab.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='humanoid_bridge', executable='bridge',
             parameters=[{'viewer': True}]),
        Node(package='humanoid_bridge', executable='wave')])
```

H. Code walkthrough

The bridge creates a supported model once. It requires exact joint-name ordering and a complete finite vector. Incoming header timestamps are checked against the ROS system clock; the watchdog uses a monotonic clock. This minimal example assumes same-host clocks and does not enable simulated ROS time. Do not set use_sim_time without adding a /clock publisher and revising the timestamp contract.

A 10 ms timer advances five 2 ms physics steps and publishes feedback. The nominal wall rate is 100 Hz, not a guaranteed physical deadline. The wave node captures the first feedback pose, then publishes a smooth 50 Hz elbow target. Stopping publication causes the bridge to request its home pose after 100 ms. That is a simulation teaching fallback, not a physical emergency stop.

JointState effort here is simulated actuator force for gear-one hinge motors. A physical driver must document whether its reported value is command torque, current-derived torque, or independently

measured output effort. Use SensorDataQoS for appropriate high-rate lossy telemetry only after checking compatibility; use short queues and expiration for commands even if reliable QoS is chosen.

I. Expected result

Two nodes appear, the left elbow moves in the simulation viewer, and /joint_states updates near the nominal 100 Hz wall timer. There is no TF publisher in this minimal bridge; RViz requires robot_state_publisher with the pinned URDF and matching mesh resolution as an extension.

J. Validation

This edition's ROS package was syntax-checked but not executed against a local Jazzy runtime. On your machine verify build, discovery, topic counts, actual viewer movement, stale-target rejection and fallback after the command node stops. Record the measured topic rate; a timer setting alone is not evidence. Then test a zero or old header stamp and a reordered name array.

K. Troubleshooting

No rclpy import usually means the wrong Python/venv. No package discovery suggests the workspace was not built or sourced. Invisible topics across terminals may indicate ROS_DOMAIN_ID or middleware configuration differences. Mesh failures in RViz are a separate URDF path issue. An idle bridge can be correct if command timestamps are rejected.

L. Practical exercise

Add a diagnostic topic reporting command age, rejected-message count and simulation real-time factor. Then place the bridge behind a lifecycle activation state so it refuses targets until configured.

M. Connection to the physical Asimov 1

Node boundaries and message contracts transfer. The physical backend needs a verified hardware interface, deterministic local execution, release-specific safety state transitions and tested power isolation. Ordinary ROS Python timers do not establish real-time performance.

N. Completion checklist

The package builds on Jazzy, two nodes exchange named data, stale targets are rejected, and stopping the publisher produces the documented simulation fallback. ROS integration remains a reader-side gate until these results are recorded.

O. Next lesson preview

Investigate support geometry and unsupported dynamics before training a policy.

Lesson 6 - Static balance and the limits of a supported pose

A. Why this matters

A robot can look upright because it is fixed to the world, balanced quasi-statically, or dynamically stabilized. Those are three different engineering situations. Begin by identifying which constraints supply the forces.

B. Concepts explained simply

Center of mass (COM) is the mass-weighted average of link centers: c=sum(m_i*c_i)/sum(m_i). Two masses, 2 kg at x=0 and 1 kg at x=.3 m, have COM x=.1 m. The support polygon is the convex hull of active contact patches. For slow, level-ground conditions, the COM's vertical projection inside that polygon is a useful static check, not a general dynamic guarantee.

Center of pressure (COP) locates the resultant normal contact force on a support plane. With two vertical loads of 100 N at x=-.1 m and 200 N at x=.1 m, COP x=.0333 m. Zero Moment Point (ZMP) is a related dynamic criterion under particular planar-contact assumptions; acceleration and angular momentum matter. COP, COM and ZMP are not interchangeable.

A linear inverted pendulum approximation uses x_ddot=(g/h)*(x-zmp) at approximately fixed COM height. At h=.6 m and x-zmp=.01 m, acceleration is about .1635 m/s^2. Small position errors can grow; being centered once is insufficient.

C. Real-world relevance

Feet may slip, lift or deform. A harness can unload the robot and change the experiment. Report support forces and constraints when claiming standing performance.

D. Architecture

Model mass and contacts -> support estimate -> COM and orientation estimate -> posture correction -> limited joint torques -> contact feedback. Static geometry informs planning; feedback and contact constraints determine dynamic recovery.

E. Prerequisites

Joint control, sensor semantics and kinematics. No physical standing tests are authorized by a simulator result.

F. Step-by-step implementation

Run the deliberately passive free-base baseline and read the JSON report. It applies no stabilizing motor commands. Its purpose is to demonstrate the failure a standing controller must improve upon.

```
python balance.py
```

Next implement supported posture tracking with the existing controller and compare to that baseline, labeling the fixture clearly. For actual unsupported standing research, use the stand task in Lesson 7 and measure it; do not silently treat a training run as solved.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/balance.py

/data/Robotics_Project/humanoid_lab/balance.py

```
"""Static support geometry and a deliberately passive free-base fall test."""
import json
import numpy as np
import mujoco as mj
from model import ROOT, load

def margin_rectangle(point, xmin, xmax, ymin, ymax):
    x, y = point
    return float(min(x-xmin, xmax-x, y-ymin, ymax-y))

def passive_trial(seed):
    rng = np.random.default_rng(seed)
    m, d = load(supported=False)
    d.qvel[:3] = rng.normal(0, .01, 3)
    start = d.qpos[:3].copy()
    fallen = False
    peak_tilt = 0.
    for _ in range(1000):
        mj.mj_step(m, d)
        rot = d.xmat[m.body('pelvis_link').id].reshape(3, 3)
        tilt = np.arccos(np.clip(rot[2, 2], -1, 1))
        peak_tilt = max(peak_tilt, float(tilt))
        if d.qpos[2] < .35 or tilt > .6:
            fallen = True; break
    return dict(seed=seed, fallen=fallen, time=float(d.time),
                displacement=float(np.linalg.norm(d.qpos[:2]-start[:2])),
                peak_tilt_rad=peak_tilt)

if __name__ == '__main__':
    # An illustrative polygon, not a measured Asimov foot polygon.
    print('Illustrative COM margin:', margin_rectangle([.01, 0], -.1, .1, -.08, .08))
    trials = [passive_trial(i) for i in range(5)]
    (ROOT/'logs/passive_trials.json').write_text(json.dumps(trials, indent=2))
    print(trials)
```

H. Code walkthrough

margin_rectangle is a signed margin to four rectangle edges for a point. Its sample rectangle is illustrative, not measured Asimov geometry. A general foot-support hull needs actual contact points and convex-polygon edge distances. passive_trial loads a floating base, introduces a small seeded initial velocity, advances up to two seconds, and labels a fall when pelvis height or tilt crosses an explicitly chosen threshold.

The world-up test uses the pelvis rotation matrix's z-axis component. Quaternion components themselves should not be thresholded as if they were tilt angles. Five seeds expose repeatability; they do not establish robust control.

I. Expected result

The prepared passive trials all fell in roughly half a simulated second. This is a useful negative baseline. A fixed-pelvis trajectory can still look stable because the fixture supplies missing support.

J. Validation

Check positive static margin inside a rectangle and negative margin outside. Evaluate any standing controller over at least 20 held-out seeds and varied initial state before widening disturbances. Proposed research gate: 60 s per run, no falls in the declared test set, bounded tracking error and no unexplained sustained torque saturation. This is a simulation gate, not a physical safety certification.

K. Troubleshooting

A zero fall rate from a fixed root is invalid evidence. Rising energy or large contact impulses may indicate poor initialization or gains. Torque saturation can make an otherwise good geometry infeasible. More friction can hide foot-slip failures instead of solving them.

L. Practical exercise

Add a small known horizontal force for a short interval in simulation. Record force magnitude, duration, impulse and exact application point, then compare recovery with an unperturbed run.

M. Connection to the physical Asimov 1

Support estimates need measured mass, contact properties and sensor calibration. A physical rig must handle the robot and a stopped or failed joint without relying on software torque.

N. Completion checklist

You can separate supported motion from static and dynamic balance, compute a simple COM/COP example, and report both successes and falls under declared conditions.

O. Next lesson preview

Train a small supported task, then extend the same environment contract to standing research.

Lesson 7 - Reinforcement learning without hidden assumptions

A. Why this matters

Learning can discover feedback strategies, but a reward is only a surrogate objective. You need enough control knowledge to recognize an unrealistic policy before deploying it.

B. Concepts explained simply

The environment advances the simulated world. The agent chooses an action from an observation. An episode begins with reset and ends with a task failure or time limit. A policy maps observations to actions; a value function estimates future reward. Training changes policy parameters; evaluation freezes them.

PPO, Proximal Policy Optimization, constrains policy updates using a clipped probability-ratio objective so updates do not move too far from the data-generating policy at once. It still needs good observations, action scales, rewards and evaluation. Stable-Baselines3 implements PPO using PyTorch; the environment implements Gymnasium's reset/step contract. [S11, S12]

For the first task the pelvis is fixed and the policy controls one elbow target offset. It observes joint errors, velocities, gravity direction, velocity channels, its previous action and the requested elbow goal.

The reward penalizes squared goal error, action changes and effort. This teaches an environment contract before introducing difficult contacts.

C. Real-world relevance

A deployable policy needs a documented observation order, normalization, action scaling, control period, actuator model and failure behavior. A neural network file alone does not specify a robot controller.

D. Architecture

Reset and randomization -> observation -> PPO actor -> bounded position target -> PD inner loop -> ten physics steps -> observation, reward, termination and metrics. Training and inference live in separate processes from hardware transport.

E. Prerequisites

Lessons 1-6, CPU PyTorch and the pinned Gymnasium/SB3 packages. Begin on CPU. A large GPU does not fix an erroneous joint order or reward.

F. Step-by-step implementation

```
python learning.py check --task reach
python learning.py train --task reach --steps 100000
python learning.py evaluate --task reach
python learning.py export --task reach
```

The environment checker verifies interface compatibility; it cannot verify that the reward represents a useful physical task. Training saves checkpoints and a final policy. Evaluation uses seeds 100-119, separate from training seed 7. Export writes a traced actor and metadata that must travel together.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/learning.py

/data/Robotics_Project/humanoid_lab/learning.py

```
"""CPU Gymnasium tasks: supported reach, free-base stand, free-base walk."""
import argparse
import json
import gymnasium as gym
import mujoco as mj
import numpy as np
from gymnasium import spaces
from model import ROOT, load, addresses, names, command

class HumanoidTask(gym.Env):
    metadata = {'render_modes': []}
    def __init__(self, task='reach'):
        super().__init__()
        if task not in ('reach', 'stand', 'walk'):
            raise ValueError(task)
        self.task = task
        self.m, self.d = load(supported=task == 'reach')
        self.ids, self.qi, self.vi = addresses(self.m)
        self.home = self.m.qpos0[self.qi].copy()
        self.elbow = names(self.m).index('left_elbow_joint')
        self.n = 1 if task == 'reach' else self.m.nu
        self.action_space = spaces.Box(-1., 1., (self.n,), np.float32)
        self.observation_space = spaces.Box(-np.inf, np.inf,
            (2*self.m.nu+9+self.n+1,), np.float32)
        self.nominal_mass = self.m.body_mass.copy()
        self.nominal_inertia = self.m.body_inertia.copy()
        self.previous = np.zeros(self.n)
        self.steps = 0
        self.goal = self.home[self.elbow]+.1

    def observation(self):
        rot = self.d.xmat[self.m.body('pelvis_link').id].reshape(3, 3)
        if self.task == 'reach':
            velocity, omega = np.zeros(3), np.zeros(3)
        else:
            # Free-joint translation velocity is world-frame; rotate to body.
            velocity = rot.T @ self.d.qvel[:3]
            omega = self.d.qvel[3:6].copy()
        return np.concatenate([self.d.qpos[self.qi]-self.home,
            self.d.qvel[self.vi], rot.T @ np.array([0., 0., -1.]),
            velocity, omega, self.previous,
            [self.goal-self.home[self.elbow]]]).astype(np.float32)

    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        mj.mj_resetData(self.m, self.d)
        # Correlated mass/inertia scaling keeps the randomized inertias physical.
        scale = self.np_random.uniform(.95, 1.05)
        self.m.body_mass[:] = self.nominal_mass*scale
        self.m.body_inertia[:] = self.nominal_inertia*scale
        mj.mj_setConst(self.m, self.d)
        self.steps = 0; self.previous = np.zeros(self.n)
        self.goal = self.home[self.elbow]+self.np_random.uniform(.05, .15)
        mj.mj_forward(self.m, self.d)
        return self.observation(), {}

    def step(self, action):
        action = np.asarray(action, dtype=float)
```

```
        if action.shape != (self.n,) or not np.isfinite(action).all():
            raise ValueError('Invalid action')
        action = np.clip(action, -1, 1)
        target = self.home.copy()
        if self.task == 'reach':
            target[self.elbow] += .2*action[0]
        else:
            target += .15*action
        for _ in range(10):
            command(self.m, self.d, target)
            mj.mj_step(self.m, self.d)
        self.steps += 1
        rot = self.d.xmat[self.m.body('pelvis_link').id].reshape(3, 3)
        upright = float(rot[2, 2])
        error = float(self.d.qpos[self.qi[self.elbow]]-self.goal)
        smooth = float(np.sum((action-self.previous)**2))
        effort = float(np.mean(self.d.ctrl**2))
        fallen = self.task != 'reach' and (self.d.qpos[2] < .35 or upright < .8)
        bad = not np.isfinite(self.d.qpos).all()
        if self.task == 'reach':
            reward = -100*error**2-.005*smooth-.0001*effort
        else:
            vx = float(self.d.qvel[0])
            desired = .15 if self.task == 'walk' else 0.
            reward = 1.+upright-np.square(vx-desired)*3-.01*smooth-.0001*effort
            if fallen: reward -= 10.
        self.previous = action.copy()
        return self.observation(), float(reward), bool(fallen or bad), self.steps >= 250, {
            'error_rad': abs(error), 'upright': upright, 'fallen': bool(fallen)}

def main():
    p = argparse.ArgumentParser()
    p.add_argument('mode', choices=['check', 'train', 'evaluate', 'export'])
    p.add_argument('--task', default='reach', choices=['reach', 'stand', 'walk'])
    p.add_argument('--steps', type=int, default=100000)
    args = p.parse_args()
    from stable_baselines3 import PPO
    from stable_baselines3.common.env_checker import check_env
    from stable_baselines3.common.monitor import Monitor
    from stable_baselines3.common.callbacks import CheckpointCallback
    folder = ROOT/'logs'/args.task; folder.mkdir(exist_ok=True)
    env = HumanoidTask(args.task)
    if args.mode == 'check':
        check_env(env); print('Environment contract passed'); return
    if args.mode == 'train':
        model = PPO('MlpPolicy', Monitor(env, str(folder/'monitor')), seed=7,
                    device='cpu', n_steps=512, batch_size=64, verbose=1)
        model.learn(total_timesteps=args.steps, callback=CheckpointCallback(
            save_freq=10000, save_path=str(folder), name_prefix='checkpoint'))
        model.save(folder/'policy')
        (folder/'metadata.json').write_text(json.dumps(dict(
            task=args.task, seed=7, action_dim=env.n,
            observation_dim=env.observation_space.shape[0],
            joints=names(env.m), dt_control=.02, mujoco=mj.__version__), indent=2))
    elif args.mode == 'evaluate':
        model = PPO.load(folder/'policy', device='cpu')
        results = []
        for seed in range(100, 120):
            obs, _ = env.reset(seed=seed); total = 0.; errors=[]
            for _ in range(250):
                action, _ = model.predict(obs, deterministic=True)
                obs, r, end, cutoff, info = env.step(action)
                total += r; errors.append(info['error_rad'])
```

```
                if end or cutoff: break
            results.append(dict(seed=seed, total=total,
                final_error=float(np.mean(errors[-50:])), **info))
        (folder/'evaluation.json').write_text(json.dumps(results, indent=2))
        print('Mean final error:', np.mean([x['final_error'] for x in results]))
        print('Fall fraction:', np.mean([x['fallen'] for x in results]))
    else:
        import torch
        model = PPO.load(folder/'policy', device='cpu')
        class Actor(torch.nn.Module):
            def __init__(self, policy):
                super().__init__(); self.policy = policy
            def forward(self, observation):
                return self.policy(observation, deterministic=True)[0]
        sample = torch.zeros((1, env.observation_space.shape[0]))
        actor = Actor(model.policy).eval()
        traced = torch.jit.trace(actor, sample)
        assert torch.allclose(actor(sample), traced(sample))
        traced.save(str(folder/'actor.pt'))
        print('Exported inference artifact; physical deployment remains gated')
    env.close()

if __name__ == '__main__':
    main()
```

H. Code walkthrough

Observation length is 2*nu+9+action_dim+1. The gravity direction comes from the pelvis rotation. Supported reach uses zero base velocity channels. The initial elbow goal is randomized over a small range. Action [-1,1] becomes a bounded .2 rad elbow offset; stand/walk use .15 rad offsets for all 25 joints. These are teaching scales, not a tuned locomotion configuration.

reset seeds the environment generator, restores state, and applies a small common mass/inertia factor. This preserves physical inertia consistency while testing a narrow kind of mismatch. It is a first domain-randomization example, not comprehensive sim-to-real randomization. step returns terminated for falls or invalid physics and truncated at 250 control steps. Treating a time limit as a fall changes value targets incorrectly.

The loop uses 2 ms physics and 20 ms policy updates. The command function supplies an ideal PD inner loop. The network learns targets, not raw motor bus messages. Export checks the traced actor against its eager output on a sample. The preparation audit also compared eager and exported actions on 100 rollout observations. SB3 predict clips Box actions; an exported actor must receive the same external action clipping. Observation-schema hash checks and replay on hardware recordings are still required before deployment.

I. Expected result

A valid environment, readable training metrics, checkpoints, metadata, a deterministic evaluation report and actor.pt. A 2,048-step smoke run only shows that the plumbing executes; it may not solve the reaching objective.

J. Validation

Measure mean absolute elbow error over the final 50 steps per held-out episode. Compare against a no-motion baseline and the classical controller with direct goal access. Proposed reach gate: mean final error below .03 rad with no nonfinite states across 20 held-out seeds. Reject a policy that wins through a sensor-order bug or simulator exploit. Rerun with multiple training seeds before claiming reproducible learning.

K. Troubleshooting

Reward improves but error does not: inspect individual reward terms and action scaling. Exploding values: verify finite observations and reward magnitude. Policy works only at reset: check whether goal information or time leaks into evaluation. Poor standing after reach success is expected; the dynamics and action dimension changed.

L. Practical exercise

Log each reward component separately. Remove the effort penalty in a simulation experiment and observe whether the policy uses more saturation to improve error. Add observation noise on copies and quantify performance loss.

M. Connection to the physical Asimov 1

The exported actor remains offline until hardware observation, timing, actuator and safety contracts pass. Randomize measured delays, friction, backlash, motor strength, noise and contact properties using defensible ranges from identification, not arbitrary values chosen to make a policy look robust.

N. Completion checklist

You can explain every observation/action element, reproduce a seeded evaluation, compare baselines, separate termination from truncation and reject reward hacking. Training throughput and wall-time estimates come from your own benchmark.

O. Next lesson preview

Extend evaluation to locomotion and learn what evidence a walking claim requires.

Lesson 8 - Standing, walking and turning as research milestones

A. Why this matters

Locomotion coordinates contact transitions while preserving feasible torques and momentum. A robot moving forward while falling is not walking, and a large reward is not a gait evaluation.

B. Concepts explained simply

A gait alternates stance, when a foot supports load, and swing, when it moves toward a future contact. A foot trajectory needs position, velocity and clearance conditions. Velocity commands specify intent, while balance and contact controllers determine whether that intent is feasible.

Classical gait generation makes phases and constraints explicit and is easier to inspect, but model error can be costly. Whole-body optimization can coordinate tasks while enforcing constraints, but requires accurate models and careful numerical design. Learned policies can handle complex contact behavior, but can exploit simulation and require broad testing. Combine approaches only with a clear authority boundary.

C. Real-world relevance

Foot slip, terrain compliance, ankle mapping and timing delays can destroy an otherwise good simulated gait. Turning adds yaw dynamics and changes support geometry; it is a separate test axis.

D. Architecture

Velocity command -> gait or learned policy -> whole-body/joint targets -> constrained feedback -> contacts -> state estimator -> gait/policy feedback. A fall detector and command watchdog remain outside the reward.

E. Prerequisites

Repeatable unsupported standing evaluation and the learned-task contract. The included stand and walk variants are a runnable research starting point, not validated locomotion solutions.

F. Step-by-step implementation

```
python learning.py check --task stand
python learning.py train --task stand --steps 100000
python learning.py evaluate --task stand
```

Do not start walking deployment until standing meets your declared gate. The walk variant rewards forward velocity near .15 m/s. It intentionally does not supply a claimed solved policy. Run it as a training experiment after standing, then inspect videos, contacts and metrics. It has no turning command in the current observation; add an explicit yaw-rate command, observation and tracking term before attempting turning training.

G. Complete code

The complete environment, training and evaluation code is learning.py in Lesson 7, with --task stand or --task walk. The following complete evaluator reads a walking trajectory CSV with columns t,x,y,yaw,vx,vy,wz,cmd_vx,cmd_vy,cmd_wz,power_w,slip_mps,fallen. Produce those channels from your simulator/controller before claiming a gait result.

```
# /data/Robotics_Project/humanoid_lab/gait_metrics.py
import argparse
import numpy as np

def summarize(path):
    x = np.genfromtxt(path, delimiter=',', names=True)
    if len(x) < 2 or not np.all(np.diff(x['t']) > 0):
        raise ValueError('Need increasing timestamped samples')
    distance = float(np.sum(np.hypot(np.diff(x['x']), np.diff(x['y']))))
    tracking = np.sqrt(np.mean((x['vx']-x['cmd_vx'])**2+
                               (x['vy']-x['cmd_vy'])**2))
    yaw_tracking = np.sqrt(np.mean((x['wz']-x['cmd_wz'])**2))
    energy = np.trapezoid(np.maximum(x['power_w'], 0), x['t'])
    return dict(distance_m=distance, velocity_rmse_mps=float(tracking),
                yaw_rate_rmse_radps=float(yaw_tracking),
                positive_energy_j=float(energy),
                slip_rms_mps=float(np.sqrt(np.mean(x['slip_mps']**2))),
                fell=bool(np.any(x['fallen'] > 0)))

if __name__ == '__main__':
    p = argparse.ArgumentParser(); p.add_argument('csv')
    print(summarize(p.parse_args().csv))
```

H. Code walkthrough

The basic stand/walk reward is intentionally small: uprightness, velocity error, action smoothness and effort. It is insufficient by itself to exclude all unwanted gaits. Add lateral velocity, yaw-rate, foot-clearance, joint-speed, contact-slip and posture constraints only after inspecting failure cases. Reward design should follow a failure explanation, not random coefficient tuning.

The metric program integrates positive power over actual time. Mechanical power tau*dq differs from battery electrical power; report which one you logged. Slip should be measured only for a declared loaded contact, not a swinging foot. A zero slip measurement with no foot load is meaningless.

I. Expected result

You obtain performance data and possibly failures. This book does not claim that the supplied reward and 100,000 steps produce standing, walking or turning. Those are gated research deliverables requiring iteration.

J. Validation

For each final policy, evaluate multiple training seeds and at least 20 held-out episodes per condition. Include still commands, starts/stops, forward/backward/lateral commands, turn commands after implemented, friction variation, modest slopes and declared push impulses. Proposed simulation gate: zero falls in the published test matrix, velocity RMSE under .1 m/s at .15 m/s command, and bounded slip and torque saturation with thresholds chosen before evaluation. Report confidence limits; zero observed falls is not zero real-world risk.

K. Troubleshooting

A policy may maximize reward by shuffling, hopping or exploiting favorable collisions. A baseline can fail because the default pose starts in penetration. Contact jitter may require a smaller timestep or better collision geometry rather than more RL steps. Do not disable self-collision solely to improve scores without documenting it.

L. Practical exercise

Add command-conditioned yaw-rate tracking and create a held-out turn test. Preserve a no-turn policy as a baseline and compare energy and fall fraction.

M. Connection to the physical Asimov 1

No simulated gait authorizes physical walking. Release-matched policy interfaces, calibrated ankles, measured timing and protected commissioning come first. Initial real tests remain supported and slow.

N. Completion checklist

Your gait claim has a defined test matrix, raw traces, videos, tracking/slip/energy metrics, fall counts and an explicit statement of model assumptions. Failed gates block progression.

O. Next lesson preview

Add perception and high-level skills without giving an AI planner unrestricted motor access.

Lesson 9 - Perception and embodied behavior

A. Why this matters

Perception should change a robot's choice of validated behavior. A detector alone is not embodied AI, while a motion script that ignores feedback is not a robust interaction.

B. Concepts explained simply

A pinhole camera maps a 3-D point to pixels: u=fx*X/Z+cx and v=fy*Y/Z+cy. With fx=400 px, cx=160 px and X/Z=.1, u=200 px. One pixel coordinate does not determine Z. A monocular colored blob supplies an image-space direction, not a verified metric reach target.

HSV separates hue from brightness and saturation, making simple color segmentation convenient under controlled lighting. Connected components group adjacent mask pixels. This is a classical vision algorithm; a trained object detector is a different technique requiring its own model license, accuracy checks and compute budget.

C. Real-world relevance

Camera intrinsics, lens distortion, exposure, lighting and camera-to-body calibration affect object estimates. Depth requires an actual depth source, known geometry or a calibrated inference method with uncertainty.

D. Architecture

Camera or test image -> calibrated preprocessing -> detector -> timestamped observation -> behavior state machine -> prevalidated skill -> command gate -> controller -> feedback. A language model may select a permitted skill name and bounded parameters, never arbitrary control code.

E. Prerequisites

Lessons 4-5, OpenCV, and a supported simulated motion. Start with synthetic images so detector failures are separable from camera transport.

F. Step-by-step implementation

```
python perception.py
```

Inspect logs/object.png and logs/detection.json. The green circle should be detected near pixel (210,110). The JSON explicitly leaves depth_m null. For a real camera extension, log its calibration ID, capture timestamp, image dimensions and detector result before connecting behavior.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/perception.py

/data/Robotics_Project/humanoid_lab/perception.py

```
"""A reproducible color detector with explicit unknown depth."""
import json
import cv2
import numpy as np
from model import ROOT

def detect(bgr):
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, np.array([40, 80, 70]), np.array([85, 255, 255]))
    count, labels, stats, centers = cv2.connectedComponentsWithStats(mask)
    if count <= 1:
        return None
    k = 1+int(np.argmax(stats[1:, cv2.CC_STAT_AREA]))
    if stats[k, cv2.CC_STAT_AREA] < 100:
        return None
    u, v = centers[k]
    return dict(u=float(u), v=float(v), depth_m=None,
                area_px=int(stats[k, cv2.CC_STAT_AREA]))

if __name__ == '__main__':
    image = np.zeros((240, 320, 3), dtype=np.uint8)
    cv2.circle(image, (210, 110), 20, (0, 255, 0), -1)
    result = detect(image)
    assert result is not None and abs(result['u']-210) < 1
    assert detect(np.zeros_like(image)) is None
    cv2.imwrite(str(ROOT/'logs/object.png'), image)
    (ROOT/'logs/detection.json').write_text(json.dumps(result, indent=2))
    print(result)
```

H. Code walkthrough

BGR-to-HSV conversion uses OpenCV's channel convention. inRange makes a binary green mask. connectedComponentsWithStats excludes background and selects the largest component above a minimum area. Returning None is an ordinary 'not observed' state. It must not be converted to a zero coordinate that looks like a valid target.

A skill library should expose small interfaces such as observe_object, supported_wave and return_to_fixture_home. Validate object identity, observation age, preconditions, amplitude and timeout before execution. The planner's output needs a strict schema and an allowlist. Unknown skills and out-of-range values are rejected. Speech recognition and natural-language planning can be added after the deterministic path passes.

I. Expected result

The synthetic positive image yields a centroid and area; the empty image yields no object; no unearned depth estimate appears. No model training or external AI service is required.

J. Validation

Test no-object, correct color, distractor color, too-small blob and multiple components. Evaluate real-image precision/recall under a declared lighting setup before enabling physical gestures. Stale detections must trigger observation refresh or no motion.

K. Troubleshooting

Wrong colors often mean RGB/BGR confusion. Hue ranges depend on OpenCV's representation. A green wall can be a larger component than the target; object identity is not guaranteed by color. Unstable detections require temporal validation and uncertainty handling, not merely larger gestures.

L. Practical exercise

Require three consistent detections over a short time window before selecting a skill. Add a disappearance test that cancels an unstarted action without inventing a new target.

M. Connection to the physical Asimov 1

The documented monocular camera is useful for direction and appearance tasks. Metric reaching requires calibrated geometry and additional constraints. Use a small supported gesture as the first demonstration rather than depth-dependent approaching.

N. Completion checklist

You can distinguish image-space detection from 3-D localization, handle missing observations, explain calibration requirements and constrain the planner to an allowlisted skill.

O. Next lesson preview

Define the contracts and rollback required to move from simulated actions to a physical stack.

Part IV - From simulation evidence to physical hardware

Lesson 10 - Sim-to-real contracts, identification and rollback

A. Why this matters

Simulation supplies perfect state, immediate commands and simplified actuators. Physical systems have stale packets, flexible structures, thermal limits and changing batteries. Deployment begins by measuring these differences and making them explicit.

B. Concepts explained simply

System identification estimates model parameters from experiments. Calibration aligns a measurement to a reference, such as encoder zero or camera geometry. Validation tests whether the calibrated model predicts new data. Domain randomization trains over a range of plausible parameters; it cannot excuse a completely wrong mechanism.

Compare a low-amplitude measured joint response against the simulator using the same sampled command. Fit only parameters the experiment can identify. For example, a static gravity test helps estimate load torque and offset but poorly identifies inertia. A slow reversal can reveal backlash or stiction; it is not a thermal test.

C. Real-world relevance

Mass distribution, friction, cable forces, motor saturation, encoder quantization, transport delay, compliance, battery voltage and foot-ground contact all change performance. A policy relying on true base velocity cannot deploy until a usable estimator supplies that signal.

D. Architecture

Simulation artifact -> schema and hash checks -> offline replay -> timing test -> hardware-in-the-loop with output inhibited -> single-joint fixture -> supported module -> protected whole body -> limited autonomous skill. Each stage has a separate go/no-go record.

E. Prerequisites

Successful numerical tests, held-out policy evaluation if a learned policy is used, release-specific hardware documentation and a reviewed physical support/stop strategy. You can complete offline work before buying actuators.

F. Step-by-step implementation

Create a deployment manifest with model/firmware/policy hashes, joint and motor orders, SI units, signed zeros, observation names, normalization constants, action scaling, controller frequency, maximum command age and fault response. Replay recorded observations through exported inference and compare with the training implementation.

Then test safety.py entirely offline. Its timestamps are supplied from one assumed monotonic clock; across hosts, redesign the protocol with measured clock synchronization, sequence handling and arrival-time constraints. Do not subtract unrelated host clocks and call the result latency.

```
python safety.py
python -m pytest -q tests/test_contracts.py
```

G. Complete code

File: /data/Robotics_Project/humanoid_lab/safety.py

/data/Robotics_Project/humanoid_lab/safety.py

```
"""Offline command-gate exercise. Not a certified hardware safety system."""
from dataclasses import dataclass
import numpy as np

@dataclass
class Gate:
    lower: np.ndarray
    upper: np.ndarray
    max_speed: float = .2
    timeout: float = .1
    last_sequence: int = -1
    fault: str = ''

    def validate(self, now, sent, sequence, target, measured, dt):
        target, measured = np.asarray(target), np.asarray(measured)
        checks = [
            (not np.isfinite([now, sent, dt]).all() or dt <= 0, 'invalid time'),
            (now-sent < 0 or now-sent > self.timeout, 'stale or future command'),
            (sequence <= self.last_sequence, 'replayed sequence'),
            (target.shape != self.lower.shape or measured.shape != target.shape,
             'shape mismatch')]
        for invalid, reason in checks:
            if invalid: self.fault = reason
        if not self.fault:
            if not np.isfinite(target).all() or not np.isfinite(measured).all():
                self.fault = 'nonfinite state'
            elif np.any(target < self.lower) or np.any(target > self.upper):
                self.fault = 'position limit'
            elif np.any(np.abs(target-measured) > self.max_speed*dt):
                self.fault = 'step envelope'
        if self.fault:
            raise RuntimeError(self.fault)
        self.last_sequence = sequence
        return target.copy()

if __name__ == '__main__':
    g = Gate(np.array([-1.]), np.array([1.]))
    print(g.validate(1., .99, 1, [.001], [0.], .01))
    try:
        g.validate(2., .99, 2, [.001], [0.], .01)
    except RuntimeError as e:
        print('Latched expected fault:', e)
```

H. Code walkthrough

Gate validates timing, sequence, shape, finiteness, range and per-command step envelope. It latches a fault, so a subsequent healthy packet cannot silently resume motion. A deliberate, reviewed reset procedure is required outside this teaching class. The class is an offline exercise, not a complete protective control system.

The step-envelope comparison does not by itself measure actual speed, and a valid position target can still demand unsafe torque. Production supervision also needs measured speed/current/temperature/voltage, travel margin, rate and acceleration limits, estimator health, missed-deadline counters and independent stop authority. Failure handling must be appropriate for a loaded robot: blindly commanding zero torque can release a limb or collapse the machine.

I. Expected result

A small valid target is accepted. An old packet raises a latched fault. No motor library is imported and no hardware output is possible.

J. Validation

Stage	Proposed go criteria	Rollback
Offline replay	Exact schema; eager/exported outputs agree on recorded dataset; no NaNs	Restore previous policy/config bundle
Timing under load	Measured worst-case inference plus transport fits reviewed deadline with margin	Reduce load or restore prior controller
HIL inhibited	Correct decoding, joint mapping, timeouts and fault state; output verified disabled	Disconnect transport and inspect logs
Single joint	Approved low-energy envelope, bounded tracking and stop response	Isolate power with load mechanically supported
Supported module	Repeatable low-speed trajectory; no thermal or deadline violations	Return to single joint and previous calibration
Whole-body support	Rig and stop tests passed; all local interfaces verified	Maintain support; isolate hazardous power
Physical skill	Repeated supervised trials within declared envelope	Disable skill; restore validated release

Record the method used to prove output inhibition and the instrument used to measure timing. A software flag saying 'disabled' is not sufficient evidence of electrical isolation.

K. Troubleshooting

Export mismatch often means normalization, action squashing, joint ordering or dtype changed. Good nominal timing with rare overruns requires stress tests and worst-case analysis. A policy that fails only after several minutes may be responding to thermal derating or voltage droop rather than perception.

L. Practical exercise

Replay duplicated, delayed, out-of-order and future-stamped commands through Gate. Confirm all expected faults are recorded and remain latched. Add a schema version mismatch case.

M. Connection to the physical Asimov 1

Ankle joint coordinates and motor A/B coordinates need a validated mechanism transform. Preserve power consistency: if joint rates equal J times motor rates, motor torques equal J transpose times joint torques under the ideal mapping. Do not reuse a small-angle approximation outside its verified range.

N. Completion checklist

You have a manifest, reproducible replay, measured timing, explicit stops and rollback procedures, and no dependency on unavailable ground-truth channels. Hardware deployment remains no-go until its own gates pass.

O. Next lesson preview

Convert an attractive platform into a complete, release-matched procurement decision.

Lesson 11 - Build a defensible EUR 6,000 budget

A. Why this matters

Actuator cost is only one part of a humanoid build. Fabrication, interfaces, tooling, protection, support, shipping and VAT can dominate the remaining budget. An unquoted line is an unknown liability, not zero euros.

B. Concepts explained simply

A bill of materials is a quantity-controlled list tied to a release. A quotation states a supplier, item revision, price, tax basis, availability, shipping and validity period. A budget allowance is an internal spending cap. These are different documents.

Landed total = parts + fabrication + shipping + applicable taxes/duties + tools/protection + spares + contingency. Avoid adding VAT twice to VAT-inclusive EU prices. Treat optional computer upgrades separately until the budget owner confirms inclusion.

C. Real-world relevance

Replacing a motor with a cheaper torque-equivalent part can change mounting, inertia, encoder semantics, protocol and thermal behavior. Compatibility is an engineering decision, not a price-filter checkbox.

D. Architecture

Select hardware release -> extract part quantities -> obtain supplier quotes -> verify compatibility -> reconcile landed costs -> reserve contingency -> purchase one stage -> reforecast remaining stages.

E. Prerequisites

Official BOM access, CAD revision, motor datasheets/protocols, firmware availability, existing tool inventory and a location suitable for a guarded fixture. The public material inspected here did not establish a complete purchasable EU BOM.

F. Step-by-step implementation

Read the itemized CSV below, keep missing prices blank, and run the audit. It deliberately reports approved=false because a known subtotal is not a feasibility total.

```
python audit.py budget config/bom.csv
```

The actuator grouping is derived from the official module checklist: three EC-A6416-P2-25, four EC-A5013-H17-100, four EC-A3814-H14-107, four EC-A4315-P2-36 and ten EC-A4310-P2-36, totaling 25. This is a procurement grouping, not a protocol specification. [S13]

A directly opened BerryBase listing showed EUR 189.00 including VAT, excluding shipping, for a Raspberry Pi 5 8 GB on 25 October 2026. It is a dated price observation, not a reserved quote. Cooling, storage, power accessories and carrier boards are additional. The supplied CSV records this single verified price; all other unresolved costs remain explicit. [S14]

G. Complete code and procurement ledger

File: /data/Robotics_Project/humanoid_lab/audit.py

/data/Robotics_Project/humanoid_lab/audit.py

```
"""Procurement completeness and recorded commissioning gate checker."""
import argparse
import csv
import json
from pathlib import Path
from model import ROOT

def budget(path):
    rows = list(csv.DictReader(Path(path).open()))
    verified = 0.; unknown=[]
    for row in rows:
        if row['verified_eur']:
            verified += float(row['quantity'])*float(row['verified_eur'])
        else:
            unknown.append(row['component'])
    return dict(known_subtotal_eur=round(verified, 2),
                unquoted=unknown, approved=False,
                reason='Incomplete quotes' if unknown else 'Requires compatibility review')

def commissioning(path):
    report = json.loads(Path(path).read_text())
    required = ['mechanical_inspection', 'polarity', 'protective_devices',
                'support_rig', 'emergency_isolation', 'watchdog',
                'encoder_direction', 'limits', 'thermal', 'rollback']
    failed = [x for x in required if report.get(x) != 'PASS']
    return dict(go=not failed, failed=failed,
                note='Record review only; not independent physical verification')

if __name__ == '__main__':
    p = argparse.ArgumentParser(); p.add_argument('mode', choices=['budget', 'commission'])
    p.add_argument('path'); args = p.parse_args()
    print(json.dumps(budget(args.path) if args.mode == 'budget'
                     else commissioning(args.path), indent=2))
```

File: /data/Robotics_Project/humanoid_lab/config/bom.csv

Item / quantity	Specification	Price evidence / supplier	Compatibility / priority
EC-A6416-P2-25 / 3	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A5013-H17-100 / 4	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A3814-H14-107 / 4	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A4315-P2-36 / 4	Exact released actuator	Unquoted / Manufacturer quote required	Revision and firmware confirm / P0
EC-A4310-P2-36 / 10	Exact released actuator	Unquoted / Manufacturer quote required	Ankle mapping required / P0
Frames and machined parts / 1	Release-specific set	Unquoted / EU fabrication quote required	CAD revision confirm / P0
Bearings fasteners shafts couplers / 1	Release-specific set	Unquoted / BOM quantities required	No substitution approved / P0
MJF printed parts / 1	PA12 set	Unquoted / EU bureau quote required	Material process match / P0
Motor drivers / 1	Confirm integrated versus additional	Unquoted / Actuator vendor	Do not double count / P0
Raspberry Pi 5 8GB / 1	Board only	EUR 189.00 / BerryBase snapshot 2026-10-25	Pi5 role verified accessories extra / P1
Radxa CM5 and custom PCBs / 1	MCB PDB head board media HAT	Unquoted / Fabrication quote required	Matched release / P0
Bench supply and distribution / 1	Voltage and regen approval pending	Unquoted / EU supplier quote required	Engineering review / P0
Battery charger protection / 1	Released matched set	Unquoted / Vendor quote required	Defer for bench platform / P2
Fuses contactor emergency stop / 1	DC-rated engineered set	Unquoted / EU supplier quote required	Rating approval pending / P0
Microcontroller CAN interface / 1	Release-specific interfaces	Unquoted / EU supplier quote required	Protocol confirmation / P0
IMU camera encoders / 1	Confirm integrated sensors first	Unquoted / EU supplier quote required	No double counting / P1
Cooling cabling connectors / 1	Release-specific set	Unquoted / EU supplier quote required	Crimp and current review / P0
Tools and metrology / 1	Required set	Unquoted / EU supplier quote required	Existing inventory credit / P0
Support rig and barriers / 1	Load-rated reviewed system	Unquoted / EU fabricator quote required	Fall arrest and access review / P0
Spare components / 1	Critical spares	Unquoted / Quote required	Same revisions / P1

Item / quantity	Specification	Price evidence / supplier	Compatibility / priority
Shipping VAT import charges / 1	Landed cost reconciliation	Unquoted / Supplier quote required	Avoid VAT double count / P0
Contingency / 1	Uncommitted reserve	Unquoted / Budget allowance	Not a supplier price / P0

The CSV quantity '1' for an assembly set is a pending quote scope, not an assertion that the set contains one fastener or one bearing. Replace set rows with actual part-number rows before ordering. Motor drivers, encoders and IMUs may be integrated: reconcile them to prevent double counting.

H. Code walkthrough

budget totals only explicitly priced quantities and reports every unquoted category. It never concludes that a robot fits a budget from a partial subtotal. commissioning uses a different path to evaluate recorded readiness; it does not verify physical tests independently.

Use priority P0 for required motion/protection dependencies, P1 for the selected research capability, and P2 for deferred expansion. Remove an optional battery from a bench stage only after choosing a suitable power/regen design; do not remove emergency isolation or support to create apparent savings.

I. Expected result

The priced board subtotal is EUR 189.00, with many unquoted categories and no purchase approval. That honest result is more useful than a fabricated EUR 5,999 full-robot estimate.

J. Validation and three scenarios

Scenario	Technical scope	Financial conclusion
A: complete Asimov 1	25-actuator machine, structure, custom boards, matched firmware, rig and tools	Not established within EUR 6,000; published USD 20,000 kit does not fit
B: lower-body research	12 actuators, pelvis, full safety/support, power and estimator	Unquoted; fewer joints do not prove affordability or safe locomotion
C: modular bench platform	One joint first, then a small supported limb or head/arm subset	Preferred staged scope; buy only after exact module quote and rig review

For Scenario C, an illustrative allocation ceiling is: EUR 1,500 actuators/drives; EUR 700 mechanical work; EUR 600 bench power/protection; EUR 400 compute/CAN/sensing; EUR 700 fixture and barriers; EUR 450 tools; EUR 300 spares; EUR 450 shipping/tax adjustments; EUR 900 contingency. Total EUR 6,000. These are planning allowances, not supplier prices or proof that the selected Asimov module fits. If the first compatible actuator exceeds its allocation, reduce scope or revise the funding decision before ordering.

A lower-body pair uses two hip-pitch, two hip-roll, two hip-yaw, two knee and four ankle actuators. Obtain twelve release-matched motor quotes and fabrication estimates before considering Scenario B. A seated or bench arm can support perception/control research without claiming biped locomotion.

K. Troubleshooting

A cheap listing may exclude the drive, encoder, shipping or tax. A STEP inventory does not encode all purchasing quantities. An available model number may ship with a different firmware. Quote validity and lead time matter as much as the price.

L. Practical exercise

Obtain quotes for one exact motor, its required interface, compatible power/protection and one mounting fixture. Record manufacturer confirmation of firmware/protocol and mounting compatibility before comparing prices.

M. Connection to the physical Asimov 1

Use the official release BOM and manufacturer confirmation to choose exact parts. EU electronics suppliers and fabrication bureaus are candidates for quotations; this edition does not assert stock or compatibility for unquoted items.

N. Completion checklist

All required rows have quantities, exact specifications, tax basis, landed cost and compatibility evidence; contingency remains funded; and the selected stage stays below its approved ceiling. Until then, stay with simulation and procurement research.

O. Next lesson preview

Inspect and fabricate mechanical parts using load paths and process constraints.

Lesson 12 - Mechanical design, fabrication and incremental assembly

A. Why this matters

The model assumes rigid links and known axes. Poor fit, weak layer orientation or a misaligned bearing changes those assumptions and may create a failure under load.

B. Concepts explained simply

STEP describes solid geometry suitable for engineering exchange. STL describes a triangle surface and normally carries neither a trustworthy unit declaration nor manufacturing tolerances. A pretty mesh is not a complete manufacturing drawing.

A load path traces force through joints, bearings, fasteners and structure into the support. A 2 kg mass at a 0.2 m lever arm produces about 3.92 Nm of static gravity torque before acceleration or impacts. An illustrative 10 Nm torque applied through a 0.05 m lever corresponds to 200 N. Those simple calculations help identify where detailed analysis is needed, but do not size a human-safe support rig.

C. Real-world relevance

Fastener preload, bearing fits, shaft alignment and cyclic fatigue affect service life. A cable crossing a joint can apply unmodeled torque or be cut by a moving edge.

D. Architecture

Released CAD and drawings -> process/material specification -> load and fit review -> first article -> dimensional inspection -> unpowered assembly -> guarded joint test -> limb -> torso integration.

E. Prerequisites

Exact release documents, competent mechanical review for load-bearing parts and rig, suitable metrology, a parts traveler and a de-energized work area. Never infer a tightening torque from the bolt diameter alone; material, grade, lubrication and joint design matter.

F. Step-by-step implementation

1. Inventory the selected module's released part numbers, drawing revisions and required quantities.
2. Trace gravity, acceleration and stop loads through the module and its fixture. Review bending, bearing and fastener loads.
3. Identify every fit and datum from drawings. Confirm units with a known dimension before exporting STL or ordering machining.
4. Fabricate one first article. Measure critical holes, bearing seats, flatness and mating distances before ordering the rest.
5. Dry-assemble unpowered. Confirm free movement, alignment, cable clearance, end stops and access for service.
6. Apply only documented assembly practices, lubrication and fastener treatments. Record torque tool identity and inspection results.
7. Reinspect after the first guarded motion and thermal cycle before expanding to a limb.

G. Complete inspection program

This lesson's executable artifact checks a simple dimensional traveler. Numerical tolerances below are examples of a file format, not approved Asimov tolerances.

```
# /data/Robotics_Project/humanoid_lab/inspect_parts.py
import csv
import sys

def inspect(path):
    failures=[]
    for row in csv.DictReader(open(path, newline='')):
        value=float(row['measured_mm'])
        lo, hi=float(row['min_mm']), float(row['max_mm'])
        if not lo <= value <= hi:
            failures.append(row['part']+':'+row['feature'])
    return failures

if __name__ == '__main__':
    failed=inspect(sys.argv[1])
    print('FAILED FEATURES:', failed)
    raise SystemExit(bool(failed))
```

CSV header: part,feature,min_mm,max_mm,measured_mm. Populate its bounds from the approved drawing, never from an illustrative tutorial number.

H. Walkthrough and material choices

The program compares measured dimensions to released bounds and returns a failing exit code for a nonconforming feature. It does not inspect material strength, surface condition or preload. Keep instrument uncertainty and calibration in the traveler.

PETG is useful for noncritical fixtures and covers when heat/creep constraints permit. ABS/ASA may suit warmer noncritical housings with appropriate printing practice. Nylon performance depends on

moisture, formulation and process. MJF PA12 has different process properties from FDM nylon. None is an automatic substitute for a released load-bearing part. Machined aluminum or steel features require the specified alloy, heat treatment, fit and finish.

FDM strength is anisotropic: a load across layer interfaces can fail differently from a load along continuous extrusion. Orientation, inserts, wall geometry and temperature affect the result. Test coupons and representative joints under the relevant load cases. Do not replace a specified 7075 component with a printed replica merely because the shape fits.

I. Expected result

A dimensionally inspected, unpowered module that moves freely through its reviewed travel without cable strain or interference. Any critical nonconformance blocks powered testing.

J. Validation

Check identification, material/process certificates where needed, drawing dimensions, bearing fit, fastener engagement, end-stop clearance, cable bend radius and fixture attachment. Measure backlash and axis alignment. Have a qualified designer determine proof-load procedures for the support; do not improvise a drop test with a robot attached.

K. Troubleshooting

Binding often indicates misalignment, warped prints or an overconstrained bearing arrangement. A loose fit should not be hidden with adhesive unless approved. Threads that strip during assembly require a design/process review, not more tightening.

L. Practical exercise

Create a serviceability drawing showing how to remove one actuator without cutting cables or unloading another unsupported structure.

M. Connection to the physical Asimov 1

The repository includes process-labeled fabrication directories. Preserve their material/process intent and reconcile part numbers to the exact assembly manual revision before manufacture.

N. Completion checklist

Critical dimensions pass; material/process choices are justified; all loads have a continuous path into a reviewed fixture; and a second person can follow the traveler.

O. Next lesson preview

Design the power and communication experiment before energizing a single actuator.

Lesson 13 - Electrical and embedded engineering on a single joint

A. Why this matters

A motor can generate torque unexpectedly, and a stopped motor can still draw substantial current. Electrical protection and mechanical support must work before software is trusted.

B. Concepts explained simply

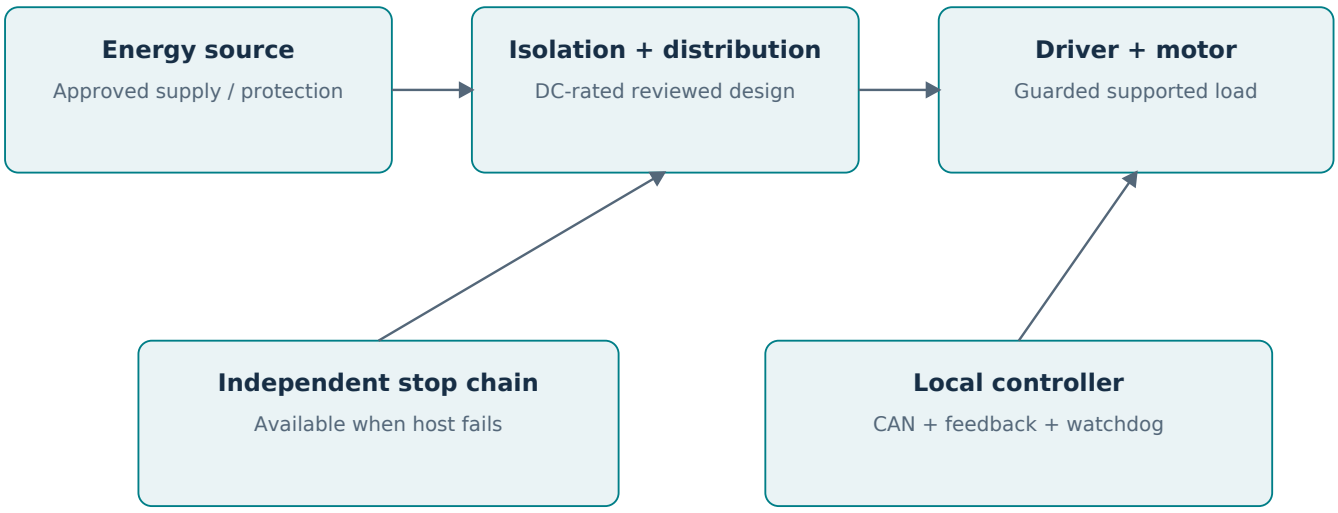
Voltage sets an electrical potential; current produces heating and magnetic force; power is V*I. A 48 V, 2 A example consumes 96 W electrically. This is an arithmetic example, not a recommended supply setting. Motor phase current and battery bus current are different quantities. Peak currents, acceleration, regeneration and duty cycle matter.

A fuse protects a circuit against specified fault conditions; it does not necessarily react fast enough to protect a person or motor semiconductor. A contactor must be rated to interrupt the actual DC circuit. An emergency-stop button is only one component of a reviewed isolation system. Regeneration can raise bus voltage when a motor is driven by a falling load; some bench supplies cannot absorb it.

C. Real-world relevance

Connector shell names do not establish pin order, polarity or current rating. The public design contains board files and wiring inventories, but a release-specific schematic and manufacturer protocol are still required for your exact assembly.

D. Architecture



The figure is a functional energy/control architecture, not a wiring schematic. Wire gauges, fuse values, contactor ratings, pinouts, precharge and regen provisions must be selected from approved hardware data and load analysis.

E. Prerequisites

Exact actuator datasheet/protocol revision, drive configuration tool, verified harness/schematic, current-limited supply approved for the actuator and regen case, suitable instruments, guarded fixture, independent isolation and competent electrical review.

F. Step-by-step implementation

1. De-energize and disconnect all energy sources. Verify isolation with a suitable instrument and protect stored-energy outputs.
2. Inspect conductor identification, polarity, insulation, strain relief, terminal retention and continuity against the approved schematic.
3. Verify supply range, current limiting, short-circuit protection, inrush/precharge and regeneration handling before attaching the actuator.
4. Support the output mechanically and clear the swept volume. Set released low-energy commissioning limits using the vendor tool before motion is possible.
5. Connect one actuator only. Read identity, firmware and feedback with torque disabled using documented commands.
6. Assign a unique ID using the official procedure. Record the physical label and bus mapping.
7. Test the independent isolation and watchdog with the mechanical load secured. Record whether the actuator actually stops producing hazardous motion.
8. Enable a small bounded motion only after direction, zero, limits and fault behavior are verified. Stop and inspect before adding another joint.

No guessed CAN payloads, voltage settings or connector pinouts are supplied. The public materials inspected do not establish a complete validated transport for your purchased release. A fabricated runnable motor command would be worse than an explicit gate.

G. Complete offline verification code

Use safety.py from Lesson 10 and audit.py from Lesson 11 to validate command envelopes and recorded readiness before any transport exists. A physical driver must be implemented from the confirmed protocol and tested on an inhibited-output fixture. The complete commissioning record template is:

/data/Robotics_Project/humanoid_lab/config/commissioning.json

```
{
  "mechanical_inspection": "NOT_TESTED",
  "polarity": "NOT_TESTED",
  "protective_devices": "NOT_TESTED",
  "support_rig": "NOT_TESTED",
  "emergency_isolation": "NOT_TESTED",
  "watchdog": "NOT_TESTED",
  "encoder_direction": "NOT_TESTED",
  "limits": "NOT_TESTED",
  "thermal": "NOT_TESTED",
  "rollback": "NOT_TESTED"
}
```

H. Walkthrough and CAN fundamentals

CAN uses a differential pair on a terminated bus. A conventional high-speed CAN segment normally has a 120-ohm termination at each physical end; two in parallel produce about 60 ohms in a de-energized resistance check when the design permits that measurement. This is a general diagnostic, not permission to add resistors without checking existing integrated termination and the release topology. Avoid arbitrary star wiring and long stubs.

Confirm bit rate, transceiver type, termination location, common-mode/ground strategy and ID uniqueness. A USB-to-CAN adapter's connector or software API does not prove protocol compatibility. A custom embedded loop should verify command age/sequence and sensor freshness locally, reject malformed data, and transition to a mechanically safe reviewed state if the host disappears.

I. Expected result

One identified actuator reports credible telemetry with its output initially disabled. The isolation and communication-failure experiments are documented before low-amplitude motion is attempted.

J. Validation

Require signed polarity/continuity inspection, supply behavior under a dummy load where appropriate, documented current limiting, unique ID, encoder-direction agreement, configuration readback, watchdog timing and emergency-isolation effect. Compare measured stop behavior against the reviewed risk envelope. A 'PASS' typed into JSON without evidence is not a test.

K. Troubleshooting

Intermittent CAN errors may indicate termination, ground, bit-rate or signal-integrity issues. Brownouts may be inrush or shared return impedance. A bus voltage spike during deceleration suggests regen handling. Stop to diagnose rather than repeatedly re-enabling a faulting drive.

L. Practical exercise

Capture an inhibited-output communication log and annotate request, response, sequence, timestamp, error flags and timeout. Confirm the same log decodes identically offline.

M. Connection to the physical Asimov 1

The manual requires assigning IDs individually before assembly. Follow its release-specific procedure; do not send a broadcast assignment on a populated robot bus. [S15]

N. Completion checklist

One joint passes identity, power, communication, feedback, limits and independent stop tests. No unresolved pinout, voltage, protocol or support assumption remains. Only then duplicate the validated module.

O. Next lesson preview

Commission a supported system while preserving local timing and safety authority.

Lesson 14 - Assembly, calibration and supported commissioning

A. Why this matters

Integration creates faults that single-component tests miss: reversed encoders, overloaded supplies, cable interference, mixed firmware and incorrect model zeros. Commissioning should isolate these faults in a controlled sequence.

B. Concepts explained simply

A zero offset aligns the encoder coordinate to the model reference pose. A direction sign aligns positive rotation. Gear ratio and parallel linkages may require a nonlinear mapping. Calibration is incomplete until a separate measurement validates the result over the intended range.

C. Real-world relevance

A biped with motor power removed may collapse. A supported fixture can reduce consequences, but it must be designed for the machine, test envelope and foreseeable failures. A tether that merely keeps the robot nearby is not necessarily a fall-arrest system.

D. Architecture

Mechanical support and isolation -> embedded protection -> real-time joint loop -> state estimator -> ROS supervision -> allowed skill. Activation requires evidence from every lower layer.

E. Prerequisites

Release-matched hardware, completed single-joint tests, load-reviewed rig, clear exclusion zone, accessible isolation, trained observer and documented shutdown/rollback procedures.

F. Step-by-step implementation

Step	Measurement and pass criterion	If it fails
Inspection	Correct parts, retained cables, fasteners and support attachment	Repair unpowered and reinspect
Power	Polarity, current limiting, protection and bus behavior match reviewed design	Disconnect and correct power design
Isolation	Each E-stop/disconnect achieves reviewed physical outcome under support	Remain no-go; repair isolation chain
Identity	Every actuator ID and firmware matches the release inventory	Test one actuator at a time
Direction	Small commanded positive change matches independent observation	Disable; correct mapping and retest
Zero	Measured reference pose matches model within approved tolerance	Recalibrate with support
Limits	Conservative position/current/speed limits read back correctly	Reconfigure while inhibited
IMU	Static orientations and zero-rate bias agree with references	Correct mounting transform/calibration
Limb motion	Small slow trajectory tracks without binding or unexplained heating	Return to joint-level investigation
Supported body	No deadline, power or mapping violations across repeated trials	Disable higher-level skill
Contact tests	Rig, foot load and contact observations match declared setup	Reassess support and model
Progression	Previous gates and reviewer approval recorded	Remain at supported stage

Set numerical tolerances from measurement uncertainty, the manufacturer and the reviewed experiment envelope. Do not use a generic 1-degree or temperature limit merely because it looks reasonable. Store the threshold and rationale with each test.

G. Complete readiness-check command

```
python audit.py commission config/commissioning.json
```

The complete program is printed in Lesson 11. Its default record is NOT_TESTED for every item, so the supplied result is no-go. Add evidence identifiers and reviewer names to a controlled record; do not replace real tests with automated approval.

H. Walkthrough and physical software boundaries

High-level compute handles ROS, perception, planning, orchestration and telemetry. The timing-critical controller validates feedback age, limits, setpoints and inference deadlines. Embedded drivers enforce local watchdogs, protection and bus semantics. Record measured cycle-time histograms and missed deadlines under realistic CPU/network load; ordinary WebSockets or MQTT are suitable for telemetry and high-level requests, not the direct stabilization loop.

Calibrate the parallel ankle in motor coordinates and joint coordinates using a verified geometric mapping and independent measurement. Validate its position and torque transforms together. A left/right sign fix in the UI must not hide an inconsistent lower-level convention.

I. Expected result

A calibrated supported module or body executes a small reviewed trajectory while feedback, temperatures, current, timing and faults are logged. You can trigger the documented stop and explain how loads remain supported.

J. Validation

Repeat each trial after a cold start, a software restart and a fault reset. Disconnect host communication intentionally with output support in place. Confirm no automatic motion resumes when packets return. Require manual re-arming through the reviewed procedure. Initial commissioning excludes unsupported dynamic walking.

K. Troubleshooting

A growing tracking error can be binding, saturation or bad zeroing. A different pose after reboot can indicate unsaved offsets or absolute-encoder interpretation. Thermal drift needs measured duty-cycle and cooling analysis. Do not hide intermittent faults by increasing watchdog timeouts without a timing justification.

L. Practical exercise

Perform an unpowered shutdown rehearsal with two people: name who isolates energy, who controls the rig, where stored energy remains, and how the faulted configuration is preserved for diagnosis.

M. Connection to the physical Asimov 1

Use the manufacturer's matched firmware/images and released procedures. A public API describing STAND/MOVE modes does not establish readiness of your assembly. Keep software compliance modes distinct from emergency isolation.

N. Completion checklist

Inspection, calibration, limits, timing, rig, isolation, watchdog and rollback all have dated evidence. A physical policy is disabled until those records pass.

O. Next lesson preview

Integrate perception with a small validated behavior and publish its evidence.

Part V - The end-to-end research demonstration

Lesson 15 - Recognize an object and execute a validated skill

A. Why this matters

An end-to-end demonstration should prove a traceable chain from observation to bounded motion and feedback. Limit the first task to the physical platform you actually have.

B. Concepts explained simply

A behavior state machine separates OBSERVE, VALIDATE, EXECUTE, VERIFY, COMPLETE and FAULT. Each transition has a condition. A skill has preconditions, bounded parameters, timeout, success criteria and cancellation behavior. 'Object seen' is not the same event as 'task complete'.

C. Real-world relevance

A one-joint supported platform can detect a colored object and make a gesture. A full-body platform with unresolved walking cannot safely promise approach or grasp. Match the demonstration to verified capability.

D. Architecture

Synthetic or camera frame -> classical detector -> permitted supported gesture -> command-envelope check -> bounded controller -> feedback log -> completion report. A future language planner may select the same gesture through a strict allowlisted schema.

E. Prerequisites

Perception tests, supported control, offline command-gate tests and logging. A physical run additionally requires every hardware commissioning gate and a verified transport implementation.

F. Step-by-step implementation

```
python perception.py
python demo.py
cat logs/demo.json
```

The included run uses a synthetic image and the fixed-pelvis Asimov model. It moves the left elbow through a smooth three-second out-and-back gesture. This completes the software chain without claiming a camera is mounted on actual hardware.

G. Complete code

File: /data/Robotics_Project/humanoid_lab/demo.py

/data/Robotics_Project/humanoid_lab/demo.py

```
"""Detect an object and execute a bounded supported elbow gesture."""
import json
import numpy as np
import mujoco as mj
from model import ROOT, load, addresses, names, command
from perception import detect
from safety import Gate

def run():
    frame = np.zeros((240, 320, 3), np.uint8)
    frame[90:130, 190:230, 1] = 255
    detection = detect(frame)
    m, d = load(); ids, qi, vi = addresses(m)
    home = d.qpos[qi].copy(); joint = names(m).index('left_elbow_joint')
    gate = Gate(m.jnt_range[ids, 0], m.jnt_range[ids, 1], max_speed=1.)
    records=[]; previous=home.copy()
    if detection is None:
        return {'state': 'NO_OBJECT', 'motion': False}
    # Prevalidated gesture only. No depth-dependent reach is attempted.
    for k in range(1500):
        t = k*m.opt.timestep
        goal = home.copy()
        goal[joint] += .05*(1-np.cos(2*np.pi*t/3.))
        approved = gate.validate(t, t, k, goal, previous, m.opt.timestep)
        command(m, d, approved); previous=approved.copy()
        mj.mj_step(m, d)
        records.append([float(d.time), float(d.qpos[qi[joint]])])
    report = {'state': 'COMPLETE_SIMULATION', 'motion': True,
              'object': detection, 'steps': len(records),
              'max_elbow_rad': max(x[1] for x in records)}
    (ROOT/'logs/demo.json').write_text(json.dumps(report, indent=2))
    np.savetxt(ROOT/'logs/demo.csv', records, delimiter=',', header='t,elbow')
    return report

if __name__ == '__main__':
    print(run())
```

H. Code walkthrough

The detector produces a centroid with unknown depth. The behavior chooses a predefined gesture, avoiding an unjustified metric reach. Each setpoint is checked against the command envelope before PD execution. The gate checks consecutive command steps in this demonstration; a separate tracking monitor must compare actual joint motion and measured speed before physical use.

The simulator logs feedback and writes a final JSON state. A physical skill implementation needs measured success criteria and an explicit FAULT/cancel path; a completed for-loop alone is not physical task verification. Extend the test with no-object input and fault injection before extending its motion.

I. Expected result

logs/demo.json reports COMPLETE_SIMULATION, 1,500 physics steps and an object observation. logs/demo.csv records elbow feedback. The prepared run's peak elbow angle was about .8843 rad; it includes the model's initial reference angle, so it is not a .8843 rad excursion.

J. Validation

Require detector positive and negative cases, finite feedback, bounded commands, final tracking-error measurement and cancellation tests. Demonstrate rejection of a stale command, invalid target and unsupported skill request. A physical first demo should pass repeated supervised trials without protection trips, unusual heating or missed-deadline violations under the same declared setup.

K. Troubleshooting

If motion occurs without a valid observation, the behavior transition is wrong. If detection succeeds but no command is sent, inspect preconditions and gate reasons. If commands are sent but tracking fails, diagnose the controller/physical layer separately from vision.

L. Practical exercise

Replace the synthetic image with a recorded, calibrated camera frame and keep the skill amplitude unchanged. Add three-frame observation consistency and a skill timeout before using a live camera stream.

M. Connection to the physical Asimov 1

Reuse the detector, state-machine design, schemas and logs. Replace only the verified simulation backend with a commissioned hardware interface; then revalidate timing, tracking, thermal behavior and stop handling. The full Asimov and a smaller module need different preconditions.

N. Completion checklist

You can trace a command to its observation, identify every gate, measure actual feedback and demonstrate a no-motion result on invalid input. Label the result as simulation, supported physical module or protected whole-body test accurately.

O. Next lesson preview

Turn a demonstration into a reproducible research result that another engineer can evaluate.

Lesson 16 - Research quality, maintenance and the next build

A. Why this matters

A working demonstration can still be fragile. Research value comes from repeatability, documented limits, baseline comparisons and an honest account of failures.

B. Concepts explained simply

Unit tests verify small properties. Integration tests verify components together. Simulation regressions detect changes in behavior under fixed conditions. Bench tests check actual interfaces. Validation measures the capability claimed; verification checks implementation against specified requirements. Both are necessary.

C. Real-world relevance

A version update, replacement actuator or changed cable routing can invalidate previous evidence. Configuration changes need explicit impact review and a path back to a known-good release.

D. Architecture

Commit -> pinned environment -> model/config hashes -> numerical tests -> simulation matrix -> release artifact -> commissioning record -> monitored deployment -> logged evidence -> issue and revision.

E. Prerequisites

All relevant earlier gates, a clear capability statement and retained raw data. Failed experiments belong in the record.

F. Step-by-step implementation

Create a release folder containing source revision, environment freeze, model hash, policy metadata, calibration IDs, test commands, numeric outputs, evaluation matrix and rollback instructions. Keep personal credentials and large raw training files out of git. Make logs immutable or uniquely named per experiment.

```
cd /data/Robotics_Project/humanoid_lab
python -m pytest -q
python learning.py check --task reach
python simulate.py --headless --seconds 10
python demo.py
python -m pip freeze > requirements-lock.txt
```

Use headless CI for numerical and fault tests. Add a separate manually controlled hardware test stage so an automatic software push cannot unexpectedly energize a mechanism. Pin CI runner images by digest for a release; do not assume a floating ubuntu-latest runner is your Ubuntu 24.04 environment.

G. Complete code

The complete numerical and contract tests are printed in Appendix A. Use them unchanged as the first regression suite. The commands above define the minimal release check; physical tests remain separately gated.

H. Walkthrough

A test asserting only that a constant equals itself adds no engineering evidence. Useful tests here check model dimensions, bounded torque, nonfinite-target rejection, IK residual, fault latching, missing depth and seeded resets. Broader tests should target changed risks: frame conversions, policy parity, transport decoding, observation age and physical limit readback.

Benchmark simulated seconds per wall second, inference latency percentiles and worst observed latency, deadline misses, memory use and log throughput. Report hardware, sample count, load, seeds and warm-up. An average latency alone hides rare overruns.

I. Expected result

Another engineer can reproduce the supported simulation demo and understand exactly which ROS, learning and physical milestones still require evidence. The completed book remains a working laboratory guide rather than a promise of a finished walking machine.

J. Validation

Perform a clean-environment rebuild, rerun the suite, reproduce one positive and one negative demonstration, and verify the rollback bundle restores the previous measured result. For every claimed capability, identify the raw evidence and the conditions where it has not been tested.

K. Troubleshooting

A result that cannot be reproduced may depend on an untracked model, seed, environment variable or calibration. A regression hidden by wider tolerances needs explanation. Restore the last good release, isolate the change and rerun the narrow relevant experiment first.

L. Practical exercise

Write a one-page research report: hypothesis, baseline, intervention, test matrix, quantitative results, failures, limits and next experiment. Make the next experiment answer one concrete uncertainty.

M. Connection to the physical Asimov 1

The same release discipline supports future limb additions and whole-body work. Re-run impact-relevant commissioning after any structural, power, firmware, estimator or policy change.

N. Completion checklist

Your demo has a precise capability statement, reproducible software, preserved logs, known limitations and tested rollback. The next build decision is grounded in measured gaps and a complete procurement forecast.

O. Next lesson preview

Choose the next research question from evidence: better state estimation, a calibrated physical joint, a more reliable skill, or contact-rich locomotion. Keep the same gates as scope increases.

Appendices - Reproducibility and evidence

Appendix A. Complete numerical and fault tests

The regression suite intentionally tests meaningful model and interface properties rather than mirroring every line of implementation. Run it from humanoid_lab with the pinned environment. These tests do not prove physical safety, successful ROS integration, learned walking or real camera performance.

File: /data/Robotics_Project/humanoid_lab/tests/test_core.py

/data/Robotics_Project/humanoid_lab/tests/test_core.py

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import numpy as np
import pytest
import mujoco as mj
from model import load, command, addresses
from kinematics import planar, solve
from balance import margin_rectangle

def test_asimov_contract():
    m, d = load()
    assert (m.nq, m.nv, m.nu) == (25, 25, 25)
    assert np.isclose(m.opt.timestep, .002)
    assert np.all(m.body_mass[1:] > 0)
    assert np.isfinite(d.qpos).all()

def test_bounded_feedback_and_rejection():
    m, d = load()
    with pytest.raises(ValueError):
        command(m, d, np.full(m.nu, np.nan))
    ids, qi, vi = addresses(m)
    for _ in range(500):
        command(m, d, m.qpos0[qi]); mj.mj_step(m, d)
    assert np.isfinite(d.qpos).all()
    assert np.max(np.abs(d.qvel)) < 50
    assert np.all(np.abs(d.ctrl) <= np.max(np.abs(m.actuator_ctrlrange), axis=1)*.25+1e-8)

def test_planar_geometry_and_ik():
    assert np.allclose(planar([0, 0]), [.5, 0])
    m, d = load()
    target = d.xpos[m.body('left_wrist_yaw_link').id].copy()+[.02, 0, .01]
    ok, error = solve(m, d, target)
    assert ok and error < .002

def test_static_margin_sign():
    assert margin_rectangle([0, 0], -.1, .1, -.1, .1) == .1
    assert margin_rectangle([.2, 0], -.1, .1, -.1, .1) < 0
```

File: /data/Robotics_Project/humanoid_lab/tests/test_contracts.py

/data/Robotics_Project/humanoid_lab/tests/test_contracts.py

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
import numpy as np
import pytest
from safety import Gate
from perception import detect
from learning import HumanoidTask

def test_fault_latches():
    g = Gate(np.array([-1.]), np.array([1.]))
    with pytest.raises(RuntimeError, match='stale'):
        g.validate(1., 0., 1, [0.], [0.], .01)
    with pytest.raises(RuntimeError):
        g.validate(1., 1., 2, [0.], [0.], .01)

def test_detection_does_not_invent_depth():
    im = np.zeros((100, 100, 3), np.uint8)
    assert detect(im) is None
    im[20:40, 30:60, 1] = 255
    assert detect(im)['depth_m'] is None

def test_seeded_reset_and_action_contract():
    env = HumanoidTask()
    a, _ = env.reset(seed=3); b, _ = env.reset(seed=3)
    assert np.array_equal(a, b)
    obs, reward, done, truncated, info = env.step(np.zeros(1))
    assert env.observation_space.contains(obs)
    assert np.isfinite(reward) and not done
    with pytest.raises(ValueError):
        env.step(np.array([np.nan]))
```

Appendix B. Measured preparation results

The following results were observed while preparing this edition, using the pinned checkout and CPU teaching environment. They are evidence about the included software, not a performance guarantee for every workstation. The initial full test suite passed seven tests. The final preparation suite and artifact checks are recorded in validation_report.json beside the manuscript.

Experiment	Measured outcome	Interpretation
Supported model load	nq=25, nv=25, nu=25	Adapter and actual asset agree on scalar coordinates
Ten-second joint run	5,000 samples, 2 ms physics step	Headless stepping/logging executes
Elbow tracking	RMSE about .01213 rad	Below the .1 rad exercise threshold
Elbow speed	Peak about .15508 rad/s	Measured simulation result, not a physical limit
Elbow torque command	Peak about 1.22718 Nm	Applied teaching command, not measured physical torque
Nearby wrist position IK	Residual about .00006504 m	Numerical position-only solve converges

Experiment	Measured outcome	Interpretation
Synthetic perception	Centroid near (210,110), unknown depth retained	Correct positive test and explicit uncertainty
Supported object gesture	1,500 physics steps, COMPLETE_SIMULATION	Perception-to-control software chain executes
Passive unsupported trials	Five of five fell around .50-.52 s	No standing capability demonstrated by passive model
PPO smoke training	2,048 timesteps, one training seed	Training pipeline check only
PPO held-out reach evaluation	Mean final error about .03744 rad over 20 seeds	Does not pass proposed .03 rad gate
Policy export	Traced actor saved; sample and 100 rollout comparisons pass	External action clipping retained; hardware replay still required
Procurement audit	EUR 189 known subtotal; many unknowns	Full-robot feasibility remains unestablished
Commissioning audit	No-go with NOT_TESTED defaults	No physical commissioning claimed
ROS integration	Source supplied and syntax inspected	Runtime integration requires reader-side Jazzy testing

The policy evaluation's zero fall fraction is trivial for supported reach. It is not a standing metric. The environment's observations and inner-loop model are idealized. Do not deploy the smoke-trained actor on hardware.

Rendered model and telemetry

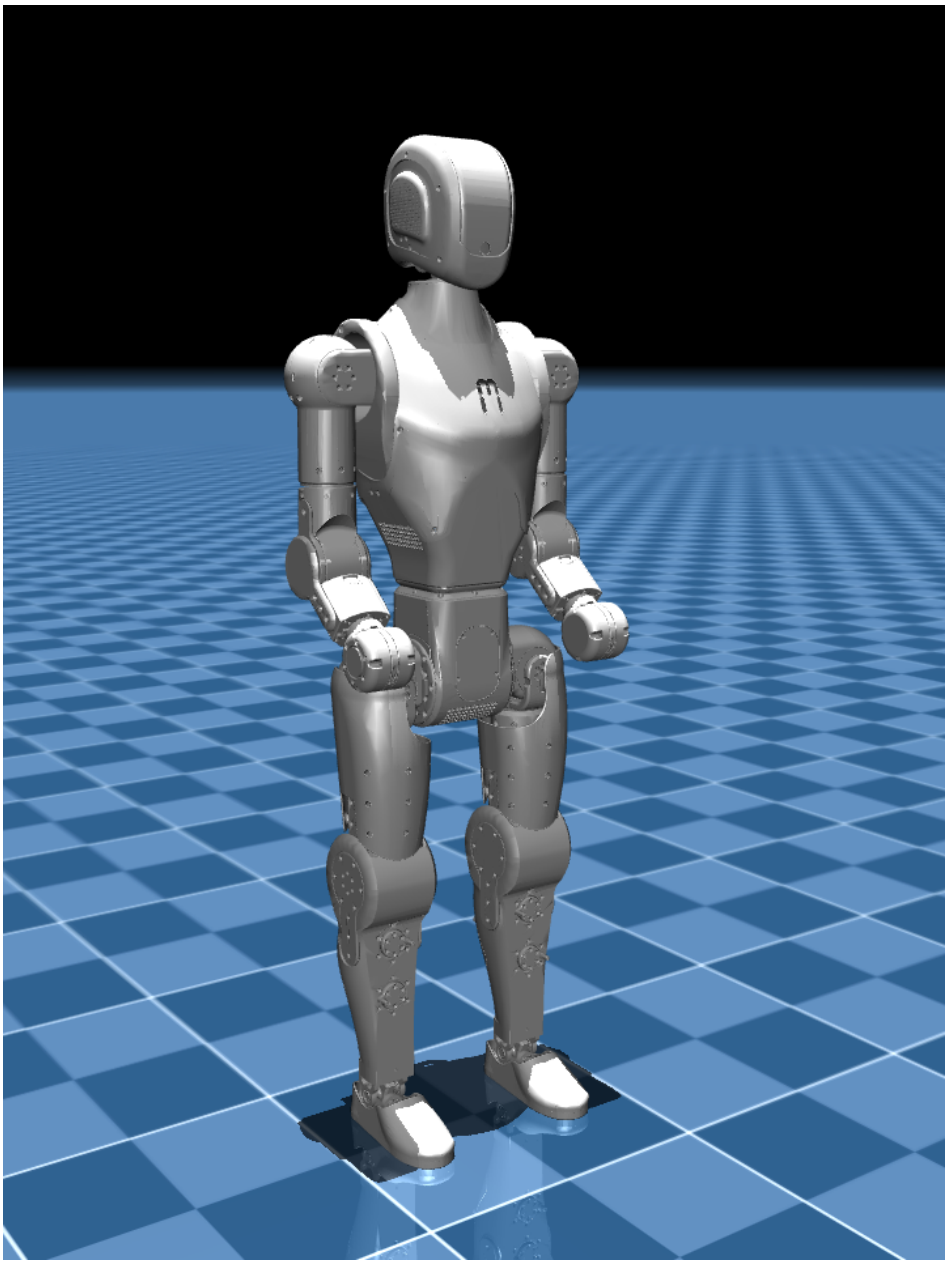


Figure B1. Actual pinned Asimov geometry rendered in the teaching fixture. Geometry comes from the upstream source under its published hardware license; this is a simulation view, not a photograph of an assembled robot.

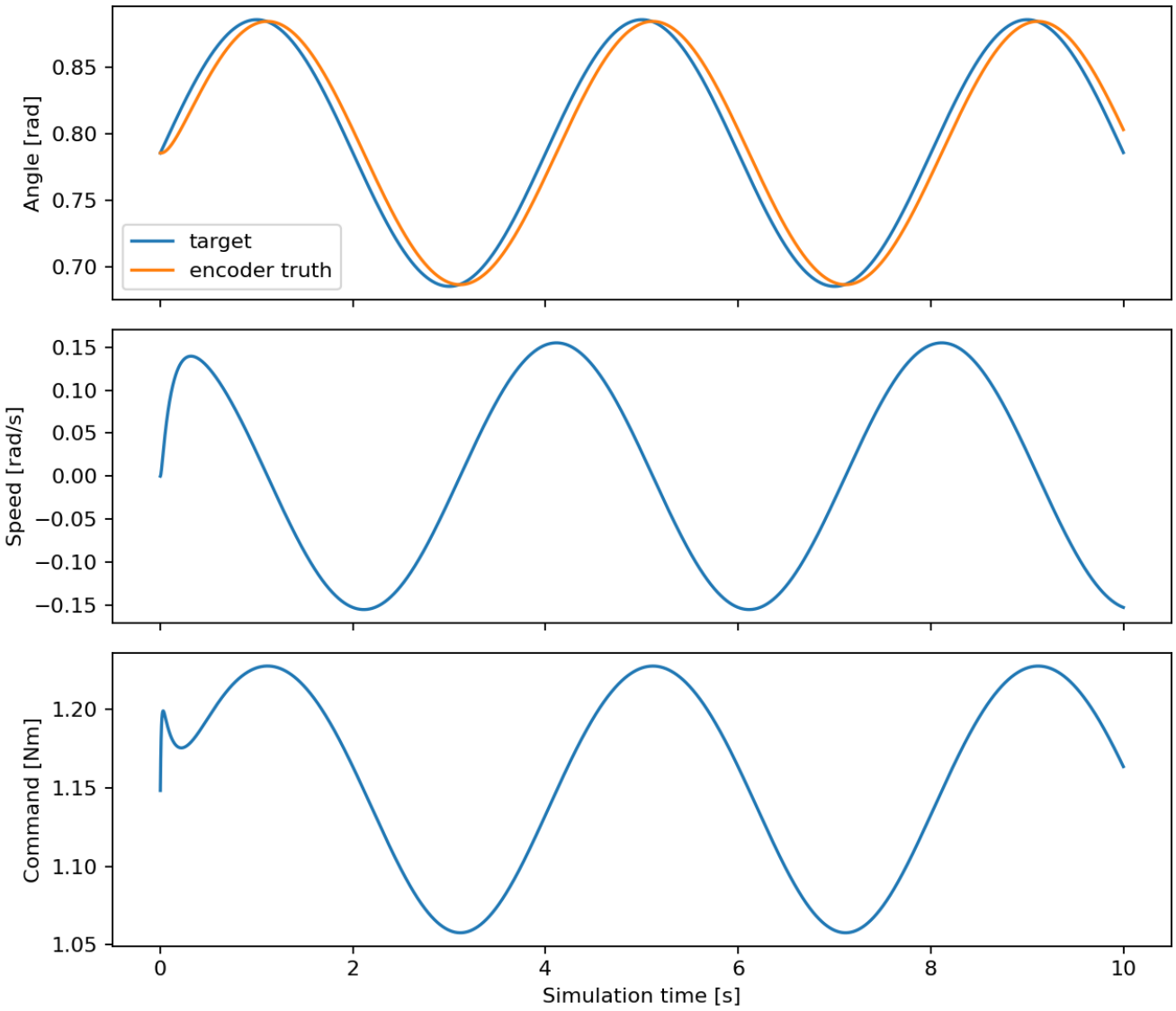


Figure B2. Ten simulated seconds of supported elbow tracking, speed and applied teaching torque. The plots are generated from the supplied CSV.

Appendix C. Live telemetry and model-rendering utilities

For a simple live dashboard, start python simulate.py --seconds 60 in one terminal and python live_dashboard.py in a second terminal from the same environment and directory. The simulator flushes its CSV periodically; the dashboard reads a recent window at 4 Hz. It is an observability path and must never become part of the actuator feedback deadline. This live GUI utility is supplied for desktop use; the preparation run verified the saved plot and offscreen model render, not an interactive desktop session.

File: /data/Robotics_Project/humanoid_lab/live_dashboard.py

/data/Robotics_Project/humanoid_lab/live_dashboard.py

```
"""Read the simulation CSV while simulate.py writes it; run in another terminal."""
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from model import ROOT
fig, axes = plt.subplots(2, 1, sharex=True)

def update(_):
    path = ROOT/'logs/joints.csv'
    if not path.exists(): return
    try:
        data = np.genfromtxt(path, delimiter=',', names=True, invalid_raise=False)
        if data.size < 2: return
        data = data[-2500:]
        axes[0].clear(); axes[1].clear()
        axes[0].plot(data['t'], data['target'], label='target')
        axes[0].plot(data['t'], data['q'], label='feedback')
        axes[0].set_ylabel('Joint angle [rad]'); axes[0].legend()
        axes[1].plot(data['t'], data['command_Nm'])
        axes[1].set_ylabel('Command [Nm]'); axes[1].set_xlabel('Simulation time [s]')
    except (ValueError, OSError, IndexError):
        return  # Writer may be between records; try next display interval.

animation = FuncAnimation(fig, update, interval=250, cache_frame_data=False)
plt.show()
```

For an offscreen geometry check on a machine with EGL support, run MUJOCO_GL=egl python render_model.py. Set the graphics backend before MuJoCo imports. If EGL fails, check the host graphics configuration; headless physics remains available. The rendered geometry was successfully produced during preparation.

File: /data/Robotics_Project/humanoid_lab/render_model.py

/data/Robotics_Project/humanoid_lab/render_model.py

```
"""Offscreen inspection image; set MUJOCO_GL before importing MuJoCo."""
import mujoco as mj
import matplotlib.pyplot as plt
from model import ROOT, load
m, d = load()
with mj.Renderer(m, height=960, width=720) as renderer:
    camera = mj.MjvCamera()
    mj.mjv_defaultCamera(camera)
    camera.lookat[:] = [0, 0, .65]
    camera.distance = 1.9
    camera.azimuth = 145
    camera.elevation = -10
    options = mj.MjvOption()
    options.geomgroup[3] = 0
    renderer.update_scene(d, camera=camera, scene_option=options)
    plt.imsave(ROOT/'logs/asimov_model.png', renderer.render())
print(ROOT/'logs/asimov_model.png')
```

Appendix D. Mermaid architecture sources

The diagrams in the book are rendered for PDF legibility. These Mermaid sources provide editable text for your project documentation.

```
flowchart TB
  Human[Human instruction] --> Planner[Task planner]
  Planner --> Skills[Validated skills]
  Skills --> Safety[Safety supervisor]
  Safety --> Controller[Joint or whole-body controller]
  Policy[Policy inference] --> Safety
  Sim[MuJoCo simulation] <--> Bridge[ROS 2 interfaces]
  Bridge <--> Controller
  Controller --> Hardware[Verified hardware abstraction]
  Hardware --> Embedded[Embedded watchdogs and drivers]
  Embedded --> Actuators[Physical actuators]
  Sensors[Sensors and estimator] --> Safety
  Sensors --> Planner
  Actuators --> Sensors
  Isolation[Independent power isolation and support] --> Actuators
  Bridge --> Logs[Telemetry and experiment records]

sequenceDiagram
  participant Human
  participant Planner
  participant Gate
  participant Controller
  participant Plant
  Human->>Planner: Request permitted skill
  Planner->>Gate: Skill name, bounded parameters, timestamp
  Gate->>Gate: Verify readiness, age, state and limits
  Gate->>Controller: Authorized trajectory
  Controller->>Plant: Bounded repeated commands
  Plant-->>Controller: Timestamped feedback
  Controller-->>Gate: Tracking and health
  Gate-->>Planner: Completed or latched fault
  Planner-->>Human: Measured result

flowchart LR
  Sensors[Encoder IMU camera contact] --> Samples[Timestamp and frame]
  Samples --> Estimator[State estimator]
  Estimator --> Control[Local feedback controller]
  Samples --> Decimator[Telemetry decimation]
  Decimator --> Dashboard[Dashboard and logs]
  Control --> Driver[Bounded actuator command]
```

Appendix E. Units, equations and glossary

Term	Meaning	Practical check
rad	Radian angle	pi rad is 180 degrees
Nm	Torque	10 N through .1 m gives 1 Nm
Hz	Updates per second	500 Hz corresponds to 2 ms
Frame	Named origin and axes	State source and destination on every transform
Quaternion	Four-number unit rotation representation	Normalize; verify component order
COM	Center of mass	Mass-weighted position; not a physical sensor
COP	Center of pressure	Requires contact-load distribution
ZMP	Zero Moment Point	Depends on model/contact assumptions
URDF	Unified Robot Description Format	Geometry/joint tree does not ensure actuator semantics
MJCF	MuJoCo model format	Contains simulator-specific dynamics/settings
QoS	Quality of service	Delivery choice does not remove message age
PD/PID	Feedback on error, rate, and optionally integral	Units and saturation must be explicit
PPO	Proximal Policy Optimization	Algorithm choice does not validate reward design
HIL	Hardware in the loop	Specify which outputs are physically inhibited
Watchdog	Deadline/freshness monitor	Test the actual fault response
E-stop	Emergency stop function	Requires reviewed physical outcome and isolation design
Sim-to-real	Transfer of tested behavior across model/hardware differences	Identification, calibration, replay and protected tests

Useful relationships: angular speed in rad/s is RPM*2*pi/60; linear lever torque is force*perpendicular distance; mechanical power is tau*dq; electrical power is V*I; center of mass is sum(m_i*c_i)/sum(m_i); rigid transform composition follows frame order. A unit conversion error is a control bug even when the program runs without exceptions.

Appendix F. Source register and update policy

Primary sources were checked for this edition on 25 October 2026. URLs may redirect or change; the local commit snapshot is the authority for file paths used by the code. Sources support specific platform or software facts, while lesson explanations, exercise parameters and proposed gates are original instructional design. Dated price observations must be rechecked before spending.

[S1] Menlo Research Asimov 1 official repository and README. Platform overview, public kit asking price, licenses and public roadmap. https://github.com/menloresearch/asimov-1/

[S2] Pinned simulation README, commit 96a6807c163e4e6cabdc124eefcc45b0ae2cb57a. Variant paths and upstream MuJoCo validation version. https://raw.githubusercontent.com/menloresearch/asimov-1/96a6807c163e4e6cabdc124eefcc45b0ae2cb57a/sim-model/README.md

[S3] Menlo mechanical system tour. Joint layout, parallel ankle distinction and document revision caveats. https://docs.menlo.ai/asimov/1/overview/system-tour/mechanical

[S4] Menlo software system tour. Software roles, release-dependent capabilities, control modes, and DAMP distinction. https://docs.menlo.ai/asimov/1/overview/system-tour/software

[S5] Menlo electrical system tour. Compute/power architecture and unresolved public CAN mapping note. https://docs.menlo.ai/asimov/1/overview/system-tour/electrical

[S6] Official ROS 2 documentation release table, raw source. Support dates including Jazzy and Lyrical. https://raw.githubusercontent.com/ros2/ros2_documentation/rolling/source/Releases.rst

[S7] Official ROS 2 Jazzy Ubuntu installation and repository configuration source. https://raw.githubusercontent.com/ros2/ros2_documentation/jazzy/source/Installation/Ubuntu-Install-Debs.rst Repository bootstrap details: https://raw.githubusercontent.com/ros2/ros2_documentation/jazzy/source/Installation/_Apt-Repositories.rst

[S8] Gazebo Harmonic documentation, recommended ROS/Gazebo pairings. https://gazebosim.org/docs/harmonic/ros_installation/

[S9] NVIDIA Isaac Sim official requirements. Read the selected release's compatibility matrix before GPU purchasing or installing Isaac Lab. https://docs.isaacsim.omniverse.nvidia.com/latest/installation/requirements.html

[S10] MuJoCo official Python documentation. Model/state APIs, stepping, viewer modes and macOS viewer requirement. https://mujoco.readthedocs.io/en/stable/python.html

[S11] Stable-Baselines3 official custom-environment documentation. Interface expectations and environment checker. https://stable-baselines3.readthedocs.io/en/master/guide/custom_env.html

[S12] Gymnasium official environment API. Reset, step, seeded randomness, termination and truncation. https://gymnasium.farama.org/api/env/

[S13] Menlo part-organization and actuator checklist. Actuator-family counts used in procurement ledger. https://docs.menlo.ai/asimov/1/build/assembly-preparations

[S14] BerryBase direct Raspberry Pi 5 8 GB product listing. EUR 189.00 VAT-inclusive observed on the preparation date, shipping extra; this is not a complete-robot quotation. https://www.berrybase.de/raspberry-pi-5-8gb-ram

[S15] Menlo official actuator-ID assignment procedure. Confirm applicability to the purchased hardware release. https://docs.menlo.ai/asimov/1/build/assembly-preparations/assign-actuator-ids

[S16] Direct upstream GitHub tree snapshot and local checkout used for the asset audit. https://api.github.com/repos/menloresearch/asimov-1/git/trees/96a6807c163e4e6cabdc124eefcc45b0ae2cb57a?recursive=1

[S17] Original 25-DOF MJCF source at the pinned revision; authoritative for this lab's modeled limits and names. https://github.com/menloresearch/asimov-1/blob/96a6807c163e4e6cabdc124eefcc45b0ae2cb57a/sim-model/asimov_1_25dof/xml/asimov_1_25dof.xml

When updating, preserve this edition and its logs. Record the new commit and diff model joints, limits, masses, collision geometry, sensors and actuator semantics. Recreate the environment, rerun regression tests, then revisit any physical calibration or safety assumption affected by the change. A newer version is an experiment to validate before it becomes a release.